Robotik Mühendisliği Serisi — Cilt 5

Algı, Kestirim ve SLAM

Kamera geometrisi, RANSAC, ICP, EKF-SLAM, poz grafiği, bacaklı durum kestirimi

Türkçe–İngilizce karma konu anlatımı
teori · sayısal örnekler · çalışan Python ve C++ kodları

Odak: bacaklı ve humanoid robotlar

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 Kamera Modeli ve Kalibrasyon **2**

1.1 İğne deliği modeli 2

1.2 DLT ile kalibrasyon 2

2 Homografi ve RANSAC **6**

2.1 Homografi 6

2.2 RANSAC 6

3 İki Görüntüden 3B: Epipolar Geometri **9**

3.1 Esas matris 9

3.2 8-nokta algoritması ve ayrıştırma 9

4 Nokta Bulutu Hizalama: Kabsch ve ICP **13**

4.1 Kabsch (eşleşmeler biliniyorsa) 13

4.2 ICP (eşleşmeler bilinmiyorsa) 13

5 EKF-SLAM **18**

5.1 Problem 18

6 Poz Grafiği ve Gauss–Newton **22**

6.1 Graf olarak SLAM 22

7 Bacaklı Robot Gövde Durum Kestirimi **27**

8 Gerçek Sistemler Haritası ve SB-1 için Algı **32**

8.1 Bu cilttekiler gerçek sistemlerde nerede? 32

8.2 Modern eğilimler (kavram düzeyinde) 32

8.3 SB-1 için gerçekçi bir algı yolu (bütçesiz) 32

8.4 Kaynaklar 33

9 Sorular **34**

10 Cevaplar **36**

Bu Cilt Hakkında

Bir robot ancak *nerede olduğunu* ve *çevresinin nasıl olduğunu* bildiği kadar iyi kontrol edilebilir. Cilt 5 bunun matematiğini kuruyor: kamera geometrisi (bir piksel dünyada neye karşılık gelir), çok görüntüden 3B, nokta bulutu hizalama, olasılıksal kestirim (EKF-SLAM ve poz grafiği) ve bacaklı robotlara özgü gövde durum kestirimi.

Hiçbir yerde OpenCV, GTSAM, Ceres gibi hazır kütüphane kullanılmadı: her algoritma NumPy veya Eigen ile 30–80 satırda sıfırdan yazıldı. Amaç, ileride bu kütüphaneleri kullanırken "içeride ne döndüğünü" bilmen. Gerçek projede tabii ki kütüphaneyi kullanacaksın (Bölüm 8).

Önkoşullar: Cilt 1 (doğrusal cebir, SVD, dönmeler, en küçük kareler, olasılık), Cilt 4B Bölüm 16–17 (Kalman filtresi). **Staj dönemi için:** telefon sürümü (Kitapçık 5) kodsuz; metin, sayılar, şekiller ve 21 soru-cevap var.

Bölüm	Konu	Kod
1	İğne deliği kamera, bozulma, DLT kalibrasyonu	`p1_camera.py`
2	Homografi, normalizasyon, RANSAC	`p2_homography.py`
3	Epipolar geometri, 8-nokta, üçgenleme, ölçek	`p3_epipolar.py`
4	Kabsch ve ICP (nokta bulutu hizalama)	`p4_icp.py/.cpp`
5	EKF-SLAM ve döngü kapatma	`p5_ekf_slam.py`
6	Poz grafiği ve Gauss–Newton (seyrek)	`p6_posegraph.py/.cpp`
7	Bacaklı robot gövde durum kestirimi	`p7_legged_est.py`, `p7_kf.cpp`
8	Gerçek sistemler haritası ve SB-1 için algı	–
9–10	Sorular ve cevaplar	–

Rastgele sayılar hakkında

Python (NumPy) ve C++ (`std::mt19937`) farklı rastgele sayı üreteçleri kullanıyor; aynı tohum (seed) farklı gürültü verir. Bu yüzden bu ciltte iki dilin sayıları *aynı mertebede* ama basamak basamak aynı değil. Kestirimde tek bir koşu "şanslı" veya "şanssız" olabilir; sonuçları her zaman birkaç tohumla dene.

Bölüm 1

Kamera Modeli ve Kalibrasyon

1.1 İğne deliği modeli

Kamera çerçevesinde (X, Y, Z) noktası, görüntü düzleminde $x = X/Z$, $y = Y/Z$ normalize koordinatlarına, oradan piksele düşer:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} \mathbf{X} \\ 1 \end{bmatrix}, \qquad \mathbf{K} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

K iç (intrinsic) parametreler: odak uzaklığı (piksel cinsinden), ana nokta (c_x, c_y), eğiklik $s \approx 0$. $(\mathbf{R}, \mathbf{t})$ **dış** (extrinsic): dünyadan kameraya dönüşüm. "$\sim$" ölçeğe kadar eşitlik: üçüncü bileşene bölünür. **Derinlik kaybolur**: tek bir pikselden noktanın uzaklığını bilemezsin, sadece bir ışın elde edersin.

Benzer üçgenlerle türetme. Kamera merkezi orijinde, optik eksen $+z$, görüntü düzlemi $z = f$'de (fiziksel olarak merkezin arkasında, ama önünde düşünmek işaretleri kolaylaştırır). Nokta (X, Y, Z)'den merkeze giden ışın, görüntü düzlemini $(fX/Z,\ fY/Z)$'de keser: kenar oranları $x/f = X/Z$. Piksellere geçiş: düzlem koordinatı (metre) piksel boyutuna bölünür ve görüntünün sol üst köşesine göre ötelenir: $u = f_x X/Z + c_x$, $v = f_y Y/Z + c_y$ ($f_x = f/$piksel genişliği). Bunu homojen biçimde yazınca $\mathbf{K}$ çıkar: $Z(u, v, 1)^T = \mathbf{K}(X, Y, Z)^T$. Dünya çerçevesindeki bir nokta önce $\mathbf{RX} + \mathbf{t}$ ile kamera çerçevesine geçer.

Lens bozulması. Gerçek lensler düz çizgileri eğer. En basit radyal model: $\mathbf{x}_d = \mathbf{x}(1 + k_1 r^2 + k_2 r^4)$, $r^2 = x^2 + y^2$. $k_1 < 0$ fıçı (geniş açılı kameralar), $k_1 > 0$ yastık. Etki merkezden uzaklaştıkça büyür.

Sayılar: $f = 500$, 640×480

$(0.2,\ -0.1,\ 2.0)$ m noktası: $u = 320 + 500 \cdot 0.1 = 370$, $v = 240 + 500 \cdot (-0.05) = 215$. $k_1 = -0.2$ bozulmasıyla $(369.88,\ 215.06)$: merkeze yakın, etki 0.1 piksel. Köşeye yakın $(0.5,\ 0.4,\ 2.0)$: $(445,\ 340) \to (442.4,\ 338.0)$, yaklaşık 3 piksel. **Çözünürlük:** 2 m'de bir piksel $2/500 = 4$ mm; 5 m'de 10 mm.

1.2 DLT ile kalibrasyon

Konumu bilinen $N \geq 6$ 3B nokta ve görüntüleri varsa, $\mathbf{P} = \mathbf{K}[\mathbf{R}\ \mathbf{t}]$ (3×4, 11 serbestlik) doğrusal olarak bulunur. Her eşleşme, $\mathbf{P}$'nin 12 elemanında doğrusal 2 denklem verir:

$$\begin{bmatrix} \mathbf{X}^T & \mathbf{0}^T & -u\mathbf{X}^T \\ \mathbf{0}^T & \mathbf{X}^T & -v\mathbf{X}^T \end{bmatrix} \mathrm{vec}(\mathbf{P}) = \mathbf{0}.$$

Bu satırlar nereden geliyor? $\mathbf{P}$'nin satırları $\mathbf{p}_1^T, \mathbf{p}_2^T, \mathbf{p}_3^T$ ise $u = \frac{\mathbf{p}_1^T \tilde{\mathbf{X}}}{\mathbf{p}_3^T \tilde{\mathbf{X}}}$ ($\tilde{\mathbf{X}}$ homojen). İçler–dışlar çarpımı: $\mathbf{p}_1^T \tilde{\mathbf{X}} - u\, \mathbf{p}_3^T \tilde{\mathbf{X}} = 0$; v için aynısı. Bilinmeyen $\mathbf{P}$ elemanlarında *doğrusal*. 12 bilinmeyen, ölçek serbest: 11; her nokta 2 denklem: en az 6 nokta. Noktalar tek bir düzlemde olmamalı (o

zaman $\mathbf{P}$'nin bir sütunu belirsiz kalır; düzlem için Bölüm 2'deki homografi kullanılır). Hepsini $\mathbf{A}\,\mathrm{vec}(\mathbf{P}) = 0$ olarak yığ; çözüm $\mathbf{A}$'nın en küçük tekil değerine ait sağ tekil vektör (Cilt 1 SVD). Sonra $\mathbf{P}$'nin sol 3×3 bloğu RQ ayrışımıyla $\mathbf{KR}$'ye ayrılır.

Pratikte **Zhang yöntemi** (OpenCV `calibrateCamera`) kullanılır: düz bir dama tahtasının farklı açılardan 10–20 fotoğrafı, her biri için bir homografi (Bölüm 2), ardından tüm parametreler ve bozulma birlikte doğrusal olmayan en küçük karelerle (*yeniden izdüşüm hatası*) iyileştirilir. İyi kalibrasyon: yeniden izdüşüm RMS'i 0.3 pikselin altında.

Sayılar: 30 nokta, 0.5 piksel gürültü

DLT'nin bulduğu $f_x = 496.3$ (gerçek 500), $c = (324.0,\ 247.7)$ (gerçek $(320,\ 240)$), dönme hatası 0.95°, yeniden izdüşüm RMS'i 0.61 piksel. Gürültü büyüdükçe f_x hatası (50 denemenin ortalaması): $\sigma = 0.1 \to 0.7$, $0.5 \to 4.0$, $2 \to 16.4$, $5 \to 44.3$ piksel; kabaca gürültüyle doğrusal.

Python

```python
import numpy as np
rng = np.random.default_rng(1)

# İğne deliği (pinhole) kamera: x_piksel ~ K [R | t] X_dünya
# SB-1'in başına takılı ucuz bir 640x480 kamera düşünelim.
fx, fy, cx, cy = 500.0, 500.0, 320.0, 240.0
K = np.array([[fx, 0, cx], [0, fy, cy], [0, 0, 1.0]])

def rotz(a):
    c, s = np.cos(a), np.sin(a); return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])
def rotx(a):
    c, s = np.cos(a), np.sin(a); return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])

def project(K, R, t, X, k1=0.0):
    """X: (N,3) dünya noktaları -> (N,2) piksel. k1: radyal bozulma katsayısı."""
    Xc = X @ R.T + t                        # kamera çerçevesi
    xn = Xc[:, :2] / Xc[:, 2:3]             # normalize görüntü düzlemi (z =1)
    r2 = (xn**2).sum(1, keepdims=True)
    xd = xn * (1 + k1 * r2)                 # basit radyal bozulma
    return xd @ K[:2, :2].T + K[:2, 2]

# 1) Tek nokta, elle kontrol edilebilir
R0, t0 = np.eye(3), np.zeros(3)
print("nokta (0.2, -0.1, 2.0) m ->", project(K, R0, t0, np.array([[0.2, -0.1, 2.0]]))[0], "piksel")
print("aynı nokta, k1 =-0.2 bozulma ->", project(K, R0, t0, np.array([[0.2, -0.1, 2.0]]),
k1=-0.2)[0].round(2), "(merkeze yakın: etki küçük)")
print("köşe noktası (0.5, 0.4, 2.0): bozulmasız", project(K, R0, t0, np.array([[0.5, 0.4,
2.0]]))[0].round(2),
      " k1 =-0.2:", project(K, R0, t0, np.array([[0.5, 0.4, 2.0]]), k1=-0.2)[0].round(2))
print("1 piksel, 2 m'de kaç mm? ", 2.0 / fx * 1000, "mm")

# 2) DLT ile kamera kalibrasyonu (P = K[R|t], 3x4) : bilinen 3B noktalar + gözlenen pikseller
Rt = rotx(-0.3) @ rotz(0.2); tt = np.array([0.1, -0.05, 1.5])
Xw = np.c_[rng.uniform(-0.5, 0.5, 30), rng.uniform(-0.5, 0.5, 30), rng.uniform(0.0, 0.6, 30)]
uv = project(K, Rt, tt, Xw) + rng.normal(0, 0.5, (30, 2))   # 0.5 piksel gürültü

def dlt_P(X, uv):
    A = []
    for (x, y, z), (u, v) in zip(X, uv):
        Xh = [x, y, z, 1]
        A.append([*Xh, 0, 0, 0, 0, *(-u * np.array(Xh))])
```

```
        A.append([0, 0, 0, 0, *Xh, *(-v * np.array(Xh))])
    _, _, Vt = np.linalg.svd(np.array(A))
    return Vt[-1].reshape(3, 4)            # en küçük tekil değere ait vektör

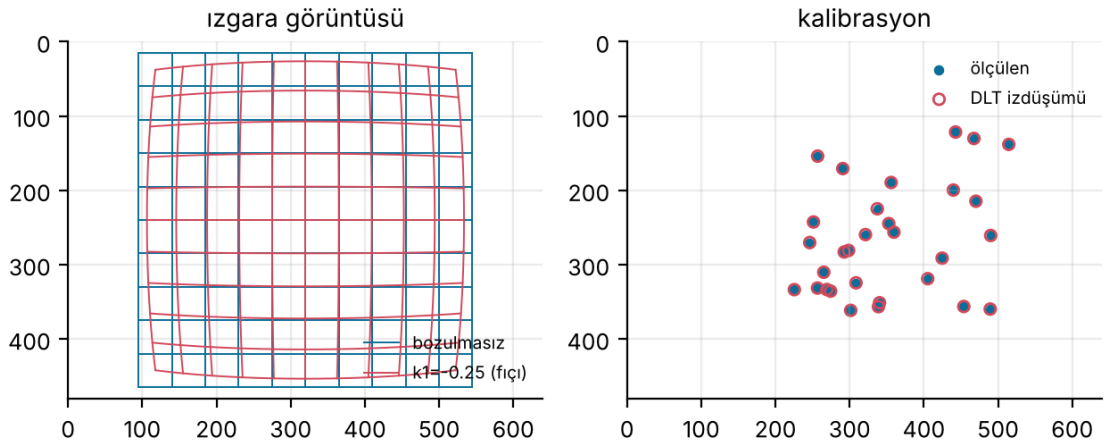
def rq3(M):
    """M = K R (K üst üçgen, R dönme) ayrışımı."""
    P = np.flipud(np.eye(3))
    Q, Rm = np.linalg.qr((P @ M).T)
    Kh = P @ Rm.T @ P; Rh = P @ Q.T
    S = np.diag(np.sign(np.diag(Kh))); Kh = Kh @ S; Rh = S @ Rh
    return Kh / Kh[2, 2], Rh

P = dlt_P(Xw, uv)
Kest, Rest = rq3(P[:, :3])
if np.linalg.det(Rest) < 0: Rest = -Rest; P = -P
test = np.c_[Xw, np.ones(30)] @ P.T; test = test[:, :2] / test[:, 2:]
print("DLT ile tahmin edilen K:\n", Kest.round(1))
print("dönme hatası (derece):", np.degrees(np.arccos(np.clip((np.trace(Rest.T @ Rt) - 1) / 2, -1,
1))).round(3))
print("yeniden izdüşüm hatası RMS:", np.sqrt(((test - uv)**2).sum(1).mean()).round(3), "piksel")

# 3) Gürültü arttıkça K tahmini nasıl bozuluyor?
for s in [0.1, 0.5, 2.0, 5.0]:
    errs = []
    for k in range(50):
        uvn = project(K, Rt, tt, Xw) + rng.normal(0, s, (30, 2))
        Ke, _ = rq3(dlt_P(Xw, uvn)[:, :3]); errs.append(abs(Ke[0, 0] - fx))
    print(f"gürültü σ ={s:3.1f} px -> |fx hata| ortalama {np.mean(errs):6.2f} px")
```

Çıktı:

```
nokta (0.2, -0.1, 2.0) m -> [370. 215.] piksel
aynı nokta, k1 =-0.2 bozulma -> [369.88 215.06] (merkeze yakın: etki küçük)
köşe noktası (0.5, 0.4, 2.0): bozulmasız [445. 340.]  k1 =-0.2: [442.44 337.95]
1 piksel, 2 m'de kaç mm?  4.0 mm
DLT ile tahmin edilen K:
 [[496.3   1.6 324. ]
 [  0.  496.2 247.7]
 [  0.    0.    1. ]]
dönme hatası (derece): 0.948
yeniden izdüşüm hatası RMS: 0.613 piksel
gürültü σ =0.1 px -> |fx hata| ortalama   0.71 px
gürültü σ =0.5 px -> |fx hata| ortalama   3.98 px
gürültü σ =2.0 px -> |fx hata| ortalama  16.38 px
gürültü σ =5.0 px -> |fx hata| ortalama  44.29 px
```



Şekil 1.1: Sol: düzgün bir ızgaranın bozulmasız (mavi) ve fıçı bozulmalı (kırmızı) görüntüsü. Sağ: kalibrasyon noktaları ve DLT ile yeniden izdüşümü.

Python notu: SVD ile homojen sistem

`_, _, Vt = np.linalg.svd(A); Vt[-1]`: $\|\mathbf{A}\mathbf{x}\|$'i $\|\mathbf{x}\| = 1$ koşuluyla en küçükleyen $\mathbf{x}$. Bu kalıbı bu ciltte dört kez göreceksin (DLT, homografi, 8-nokta, üçgenleme): hepsi aynı "doğrusal denklemleri yığ, SVD'nin son satırını al" fikri.

Bölüm 2

Homografi ve RANSAC

2.1 Homografi

Sahne düzlemselse (zemin, duvar, dama tahtası, AprilTag), iki görüntü arasındaki ilişki bir 3×3 matristir: $\mathbf{x}' \sim \mathbf{H}\mathbf{x}$. Ölçek önemsiz olduğu için 8 serbestlik; her eşleşme 2 denklem: **en az 4 nokta**. Ayrıca kamera yalnızca dönüyorsa (öteleme yok) sahne düzlemsel olmasa da ilişki bir homografidir (panorama). Kullanımlar: AprilTag pozunun bulunması, zemin düzleminin "kuşbakışı" görüntüsü, Zhang kalibrasyonu.

Neden düzlemde bir matris? Düzlemi $Z = 0$ seç. $\mathbf{P}\tilde{\mathbf{X}} = \mathbf{K}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{r}_3\ \mathbf{t}](X, Y, 0, 1)^T = \mathbf{K}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}](X, Y, 1)^T$: üçüncü sütun düşer, kalan 3×3 matris $\mathbf{H} = \mathbf{K}[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$. İki farklı kameranın aynı düzleme bakan homografileri $\mathbf{H}_1$, $\mathbf{H}_2$ ise görüntüler arası ilişki $\mathbf{H}_2\mathbf{H}_1^{-1}$: yine bir homografi. Tersine, $\mathbf{K}$ biliniyorsa $\mathbf{H}$'den $\mathbf{r}_1, \mathbf{r}_2, \mathbf{t}$ okunur ve $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2$: AprilTag'in poz kestirmesi tam olarak budur.

DLT ile çözüm Bölüm 1'deki gibi. **Hartley normalizasyonu:** noktaları merkezleri 0'da, ortalama uzaklıkları $\sqrt{2}$ olacak şekilde ölçekle, çöz, geri dönüştür ($\mathbf{H} = \mathbf{T}_2^{-1}\tilde{\mathbf{H}}\mathbf{T}_1$). Piksel koordinatları (yüzlerce) ile 1'ler aynı matriste karışınca SVD kötü koşullanır.

2.2 RANSAC

Özellik eşleştirme (ORB, SIFT) eşleşmelerin %20–50'sini yanlış verebilir. En küçük kareler bir tek kötü noktayla bile bozulur. **RANSAC**:

1. Rastgele en küçük örneği seç (homografi için 4).
2. Modeli kestir; hatası eşikten küçük olan noktaları say (iç noktalar, *inliers*).
3. En çok iç noktası olan modeli tut; sonunda tüm iç noktalarla yeniden kestir.

İterasyon formülünün türetmesi. Rastgele s eşleşmenin hepsinin iç nokta olma olasılığı w^s (yaklaşık bağımsız seçim). Bir denemenin başarısız olma olasılığı $1 - w^s$; k denemenin hepsinin başarısız olması $(1 - w^s)^k$. Bunun $1 - p$ olmasını iste ve log al: $k \log(1 - w^s) = \log(1 - p)$. Gereken iterasyon: iç oran w, örnek boyutu s, başarı olasılığı p için $k = \frac{\log(1-p)}{\log(1-w^s)}$.

Sayılar: 100 eşleşme, %40'ı yanlış

Hepsiyle DLT: test noktalarında ortalama hata **460 piksel** (tamamen bozuk). RANSAC (200 iterasyon, 3 piksel eşik): **0.34 piksel**; 60 gerçek iç noktanın 58'ini buldu. Normalizasyonun etkisi bu örnekte küçük (0.35 → 0.27 piksel) ama Bölüm 3'te dramatik.
İterasyon sayısı ($p = 0.99$): $w = 0.9$: 5 ($s = 4$), 9 ($s = 8$). $w = 0.6$: 34 / 272. $w = 0.3$: 567 / **70 188**. Bu yüzden **en küçük örneği** kullanan çözücüler (5-nokta, 3-nokta PnP) değerlidir.

Python

```python
import numpy as np
rng = np.random.default_rng(2)

# Homografi: düzlemsel bir sahnenin (zemin, duvar, AprilTag) iki görüntüsü arasında x' ~ H x  (3x3, 8 serbestlik)
def normalize(p):
    """Hartley normalizasyonu: merkezi 0'a, ortalama uzaklığı sqrt(2)'ye taşı."""
    c = p.mean(0); d = np.sqrt(((p - c)**2).sum(1)).mean()
    s = np.sqrt(2) / d
    T = np.array([[s, 0, -s * c[0]], [0, s, -s * c[1]], [0, 0, 1]])
    return (np.c_[p, np.ones(len(p))] @ T.T)[:, :2], T

def dlt_H(p, q, norm=True):
    if norm:
        pn, T1 = normalize(p); qn, T2 = normalize(q)
    else:
        pn, qn, T1, T2 = p, q, np.eye(3), np.eye(3)
    A = []
    for (x, y), (u, v) in zip(pn, qn):
        A.append([-x, -y, -1, 0, 0, 0, u * x, u * y, u])
        A.append([0, 0, 0, -x, -y, -1, v * x, v * y, v])
    _, _, Vt = np.linalg.svd(np.array(A))
    Hn = Vt[-1].reshape(3, 3)
    H = np.linalg.inv(T2) @ Hn @ T1
    return H / H[2, 2]

def apply(H, p):
    q = np.c_[p, np.ones(len(p))] @ H.T
    return q[:, :2] / q[:, 2:]

H_true = np.array([[1.10, 0.05, 30.0], [-0.08, 0.95, 12.0], [2e-4, -1e-4, 1.0]])

# 1) En az 4 nokta: tam çözüm
p4 = np.array([[100, 100], [500, 120], [480, 400], [120, 380.0]])
H4 = dlt_H(p4, apply(H_true, p4))
print("4 noktadan H (gerçekle fark, en büyük):", np.abs(H4 - H_true).max().round(8))

# 2) Normalizasyonun önemi: 50 nokta, 1 piksel gürültü
p = rng.uniform([0, 0], [640, 480], (50, 2)); q = apply(H_true, p) + rng.normal(0, 1.0, (50, 2))
pt = rng.uniform([0, 0], [640, 480], (200, 2))
for nm in [False, True]:
    H = dlt_H(p, q, nm)
    e = np.sqrt(((apply(H, pt) - apply(H_true, pt))**2).sum(1)).mean()
    print(f"normalizasyon ={'evet' if nm else 'hayır'}: test noktalarında ortalama hata {e:.3f} px")

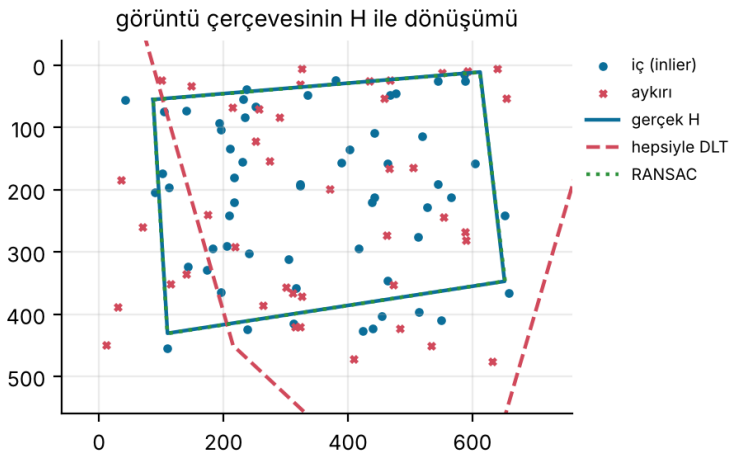
# 3) RANSAC: eşleşmelerin %40'ı yanlış (özellik eşleştirmede olağan)
N = 100; out = int(0.4 * N)
p = rng.uniform([0, 0], [640, 480], (N, 2)); q = apply(H_true, p) + rng.normal(0, 1.0, (N, 2))
q[:out] = rng.uniform([0, 0], [640, 480], (out, 2))        # aykırı (outlier) eşleşmeler
H_naive = dlt_H(p, q)
def ransac_H(p, q, thr=3.0, iters=200):
    best = np.zeros(len(p), bool)
    for _ in range(iters):
        idx = rng.choice(len(p), 4, replace=False)
        try: H = dlt_H(p[idx], q[idx])
        except np.linalg.LinAlgError: continue
        inl = np.sqrt(((apply(H, p) - q)**2).sum(1)) < thr
        if inl.sum() > best.sum(): best = inl
    H = dlt_H(p[best], q[best])                            # tüm iç noktalarla yeniden kestir
    inl = np.sqrt(((apply(H, p) - q)**2).sum(1)) < thr      # iç noktaları bu daha iyi H ile yeniden seç
```

```
    return dlt_H(p[inl], q[inl]), inl
H_r, inl = ransac_H(p, q)
for nm, H in [("hepsiyle DLT", H_naive), ("RANSAC", H_r)]:
    e = np.sqrt(((apply(H, pt) - apply(H_true, pt))**2).sum(1)).mean()
    print(f"{nm:13s}: ortalama hata {e:9.3f} px")
print("RANSAC iç nokta sayısı:", inl.sum(), "/", N, " gerçek iç noktalardan bulunan:", inl[out:].sum(),
"/", N - out)

# Kaç iterasyon gerekir? w = iç oran, s = örnek boyutu, p = başarı olasılığı
for w in [0.9, 0.6, 0.3]:
    for s in [4, 8]:
        k = np.log(1 - 0.99) / np.log(1 - w**s)
        print(f"iç oran {w:.1f}, örnek {s}: %99 için {int(np.ceil(k)):6d} iterasyon")
```

Çıktı:

```
4 noktadan H (gerçekle fark, en büyük): 0.0
normalizasyon =hayır: test noktalarında ortalama hata 0.347 px
normalizasyon =evet: test noktalarında ortalama hata 0.268 px
hepsiyle DLT : ortalama hata   460.194 px
RANSAC       : ortalama hata     0.336 px
RANSAC iç nokta sayısı: 58 / 100  gerçek iç noktalardan bulunan: 58 / 60
iç oran 0.9, örnek 4: %99 için      5 iterasyon
iç oran 0.9, örnek 8: %99 için      9 iterasyon
iç oran 0.6, örnek 4: %99 için     34 iterasyon
iç oran 0.6, örnek 8: %99 için    272 iterasyon
iç oran 0.3, örnek 4: %99 için    567 iterasyon
iç oran 0.3, örnek 8: %99 için  70188 iterasyon
```



Şekil 2.1: Görüntü çerçevesinin gerçek, "hepsiyle DLT" ve RANSAC homografileri ile dönüşümü. RANSAC gerçek çerçeveyle çakışıyor; aykırı noktalar (x) dışarıda bırakıldı.

Bölüm 3

İki Görüntüden 3B: Epipolar Geometri

3.1 Esas matris

Kamera 1'den kamera 2'ye dönüşüm $\mathbf{X}_2 = \mathbf{R}\mathbf{X}_1 + \mathbf{t}$. Aynı noktanın iki normalize görüntü koordinatı $\mathbf{x}_1, \mathbf{x}_2$ (homojen) için

$$\mathbf{x}_2^T \mathbf{E}\, \mathbf{x}_1 = 0, \qquad \mathbf{E} = [\mathbf{t}]_\times \mathbf{R} \quad \text{(esas matris, } \textit{essential}\text{)}.$$

Geometrik anlamı: $\mathbf{x}_1$'in ışını, ikinci görüntüde bir doğruya (epipolar doğru) düşer; eşi o doğrunun üzerindedir. Piksel koordinatlarıyla aynı denklem $\mathbf{F} = \mathbf{K}^{-T}\mathbf{E}\mathbf{K}^{-1}$ (temel matris, *fundamental*) ile yazılır.

Türetme. $\mathbf{X}_2 = \mathbf{R}\mathbf{X}_1 + \mathbf{t}$. İki tarafı soldan $\mathbf{t}$ ile çapraz çarp ($\mathbf{t} \times \mathbf{t} = \mathbf{0}$): $\mathbf{t} \times \mathbf{X}_2 = \mathbf{t} \times \mathbf{R}\mathbf{X}_1$. Şimdi $\mathbf{X}_2$ ile iç çarp; sol taraf $\mathbf{X}_2 \cdot (\mathbf{t} \times \mathbf{X}_2) = 0$ (bir vektör, kendisiyle başka bir vektörün çapraz çarpımına diktir):

$$0 = \mathbf{X}_2^T [\mathbf{t}]_\times \mathbf{R}\mathbf{X}_1.$$

$\mathbf{X}_i = Z_i \mathbf{x}_i$ (derinlik çarpı normalize koordinat) koy, $Z_1 Z_2$'ye böl: $\mathbf{x}_2^T \mathbf{E} \mathbf{x}_1 = 0$. Geometrik anlam: kamera merkezleri ve 3B nokta aynı düzlemde (epipolar düzlem); üç vektör $\mathbf{X}_2$, $\mathbf{t}$, $\mathbf{R}\mathbf{X}_1$ eş düzlemli, üçlü çarpımları sıfır. **Tekil değerler:** $[\mathbf{t}]_\times$'in tekil değerleri $(|\mathbf{t}|, |\mathbf{t}|, 0)$ (bir yönü yok eder, ona dik düzlemde 90° döndürüp $|\mathbf{t}|$ ile ölçekler); $\mathbf{R}$ ile çarpmak tekil değerleri değiştirmez. Bu yüzden 8-nokta sonucu $(1, 1, 0)$'a izdüşürülür.

$\mathbf{E}$'nin 5 serbestliği var (dönme 3 + ötelemenin *yönü* 2). **Ölçek kaybı:** $\mathbf{t}$'yi 2 katına, sahneyi de 2 katına çıkarırsan görüntüler aynıdır. Tek kamerayla (monoküler) ölçek *asla* gözlemlenemez; dışarıdan gelmeli: stereo taban çizgisi, IMU, bacak odometrisi, bilinen boyutlu bir nesne.

3.2 8-nokta algoritması ve ayrıştırma

Her eşleşme $\mathbf{E}$'nin 9 elemanında bir doğrusal denklem: $(\mathbf{x}_2 \otimes \mathbf{x}_1)^T \mathrm{vec}(\mathbf{E}) = 0$. 8 eşleşme + SVD. Sonra $\mathbf{E}$'yi tekil değerleri $(1, 1, 0)$ olacak şekilde düzelt. $\mathbf{E} = \mathbf{U}\mathrm{diag}(1, 1, 0)\mathbf{V}^T$'den dört aday çıkar: $\mathbf{R} \in \{\mathbf{U}\mathbf{W}\mathbf{V}^T, \mathbf{U}\mathbf{W}^T\mathbf{V}^T\}$, $\mathbf{t} = \pm\mathbf{u}_3$. Doğrusu, noktaları *iki kameranın da önüne* düşüren tektir (*cheirality*).

Üçgenleme: iki ışının "kesişimi". $\mathbf{P}_1 = [\mathbf{I}\ \mathbf{0}]$, $\mathbf{P}_2 = [\mathbf{R}\ \mathbf{t}]$ için her görüntü 2 doğrusal denklem verir; 4 denklem, SVD.

Üçgenleme denklemleri: $\mathbf{x} \sim \mathbf{P}\tilde{\mathbf{X}}$ demek, iki vektörün paralel olması: $\mathbf{x} \times \mathbf{P}\tilde{\mathbf{X}} = \mathbf{0}$. Üç bileşenden ikisi bağımsız: $x\, \mathbf{p}_3^T\tilde{\mathbf{X}} - \mathbf{p}_1^T\tilde{\mathbf{X}} = 0$, $y\, \mathbf{p}_3^T\tilde{\mathbf{X}} - \mathbf{p}_2^T\tilde{\mathbf{X}} = 0$ (DLT ile aynı kalıp, bu sefer bilinmeyen nokta). İki kamera, 4 denklem, 4 homojen bilinmeyen: en küçük tekil vektör.

Derinlik formülünün türetmesi: stereo (paralel kameralar, aralarında yatay b): sol görüntüde $u_1 = fX/Z$, sağda $u_2 = f(X - b)/Z$; fark $d = u_1 - u_2 = fb/Z \Rightarrow Z = fb/d$. Türev $|dZ/dd| = fb/d^2 = Z^2/(fb)$, dolayısıyla $\sigma_Z \approx \frac{Z^2}{fb}\sigma_d$.

Derinlik belirsizliği (bilinen poz, taban çizgisi b): $z = fb/d$ (d: piksel farkı, *disparity*) $\Rightarrow$ $\sigma_z \approx \frac{z^2}{fb}\sigma_d$. Derinlik hatası uzaklığın *karesiyle* büyür.

Sayılar: 40 nokta, 2–5 m, 20 cm yan hareket

Öteleme yönü hatası (30 deneme medyanı):

gürültü	ham 8-nokta	Hartley normalizasyonlu
0.1 px	1.0°	1.5°
0.5 px	39.4°	6.4°
1.0 px	**84.7°**	6.6°

Normalizasyon burada hayati. Seçilen çözüm: dönme hatası 0.56°, öteleme yönü 6.2°; $|\mathbf{t}|$ tahmini 1 (ölçek yok!). Ölçeği dışarıdan verince 3B noktaların medyan hatası 0.47 m; derinliğe göre 0.23 / 0.46 / 0.71 m (2–3 / 3–4 / 4–5 m). **Poz biliniyorsa** 3.5–4.5 m'de derinlik hatası 0.068 m; formül $16 \cdot 0.5/(500 \cdot 0.2) = 0.08$ m. Fark poz kestirimindeki hatadan geliyor: bu yüzden gerçek sistemler poz ve noktaları *birlikte* iyileştirir (**bundle adjustment**, Bölüm 6'daki Gauss–Newton'un aynısı).

Python

```python
import numpy as np
rng = np.random.default_rng(3)

# İki görüntüden 3B: robot 20 cm yürüyüp biraz dönüyor; aynı noktaları iki karede görüyor.
K = np.array([[500, 0, 320], [0, 500, 240], [0, 0, 1.0]]); Ki = np.linalg.inv(K)
def roty(a):
    c, s = np.cos(a), np.sin(a); return np.array([[c, 0, s], [0, 1, 0], [-s, 0, c]])
def skew(v): return np.array([[0, -v[2], v[1]], [v[2], 0, -v[0]], [-v[1], v[0], 0]])

R = roty(0.1); t = np.array([-0.2, 0.0, 0.02])        # kamera 2 = R X1 + t
X = np.c_[rng.uniform(-1, 1, 40), rng.uniform(-0.6, 0.6, 40), rng.uniform(2, 5, 40)]
def proj(Xc): x = Xc @ K.T; return x[:, :2] / x[:, 2:]
u1 = proj(X) + rng.normal(0, 0.5, (40, 2)); u2 = proj(X @ R.T + t) + rng.normal(0, 0.5, (40, 2))
n1 = np.c_[u1, np.ones(40)] @ Ki.T; n2 = np.c_[u2, np.ones(40)] @ Ki.T   # normalize koordinatlar

E_true = skew(t) @ R
a0 = X[0] / X[0, 2]; X2 = R @ X[0] + t; b0 = X2 / X2[2]       # gürültüsüz normalize koordinatlar
print("epipolar kısıt  x2ᵀ E x1 (gürültüsüz) =", f"{b0 @ E_true @ a0:.1e}", "  (gürültülü ort. |.|:",
      f"{np.abs(np.einsum('ij,jk,ik->i', n2, E_true, n1)).mean():.1e})")

def eight_point(a, b):
    A = np.array([np.kron(bi, ai) for ai, bi in zip(a, b)])  # b^T E a = 0 -> vec(E) satırı
    _, _, Vt = np.linalg.svd(A); E = Vt[-1].reshape(3, 3)
    U, S, Vt = np.linalg.svd(E)                              # E'nin iki eşit, bir sıfır tekil değeri
    olmalı
    return U @ np.diag([1, 1, 0]) @ Vt

def hartley(p):
    c = p[:, :2].mean(0); d = np.sqrt(((p[:, :2] - c)**2).sum(1)).mean(); s = np.sqrt(2) / d
    T = np.array([[s, 0, -s * c[0]], [0, s, -s * c[1]], [0, 0, 1]]); return p @ T.T, T
def eight_point_norm(a, b):
    an, T1 = hartley(a); bn, T2 = hartley(b)
    E = T2.T @ eight_point(an, bn) @ T1                      # normalize uzayda çöz, geri dönüştür
    U, _, Vt = np.linalg.svd(E); return U @ np.diag([1, 1, 0]) @ Vt

def triangulate(P1, P2, a, b):
```

```
    A = np.array([a[0] * P1[2] - P1[0], a[1] * P1[2] - P1[1], b[0] * P2[2] - P2[0], b[1] * P2[2] - P2[1]])
    _, _, Vt = np.linalg.svd(A); Xh = Vt[-1]; return Xh[:3] / Xh[3]

def decompose(E, a, b):
    U, _, Vt = np.linalg.svd(E)
    if np.linalg.det(U) < 0: U = -U
    if np.linalg.det(Vt) < 0: Vt = -Vt
    W = np.array([[0, -1, 0], [1, 0, 0], [0, 0, 1]])
    best = None
    for Rc in [U @ W @ Vt, U @ W.T @ Vt]:
        for tc in [U[:, 2], -U[:, 2]]:
            P1 = np.c_[np.eye(3), np.zeros(3)]; P2 = np.c_[Rc, tc]
            Xs = np.array([triangulate(P1, P2, ai, bi) for ai, bi in zip(a, b)])
           front = ((Xs[:, 2] > 0) & ((Xs @ Rc.T + tc)[:, 2] > 0)).sum()   # iki kameranın da önünde mi?
            if best is None or front > best[0]: best = (front, Rc, tc, Xs)
    return best

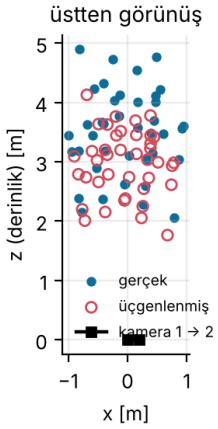
print("öteleme yönü hatası (30 deneme medyanı, derece):")
for sig in [0.1, 0.5, 1.0]:
    res = []
    for f in (eight_point, eight_point_norm):
        es = []
        for k in range(30):
            a = np.c_[proj(X) + rng.normal(0, sig, (40, 2)), np.ones(40)] @ Ki.T
            b = np.c_[proj(X @ R.T + t) + rng.normal(0, sig, (40, 2)), np.ones(40)] @ Ki.T
            _, _, tc, _ = decompose(f(a, b), a, b); es.append(np.degrees(np.arccos(abs(tc @ t) / np.linalg.norm(t))))
        res.append(np.median(es))
    print(f"  gürültü {sig:.1f} px:  ham 8-nokta {res[0]:6.2f}°   Hartley normalizasyonlu {res[1]:6.2f}°")

E = eight_point_norm(n1, n2)
front, Re, te, Xs = decompose(E, n1, n2)
ang = np.degrees(np.arccos(np.clip((np.trace(Re.T @ R) - 1) / 2, -1, 1)))
print("4 adaydan seçilen: iki kameranın önünde kalan nokta sayısı", front, "/ 40")
print(f"dönme hatası {ang:.3f}°;  öteleme yönü hatası {np.degrees(np.arccos(abs(te @ t) / np.linalg.norm(t))):.3f}°")
print("|t| tahmini:", np.linalg.norm(te).round(3), " (gerçek 0.201) -> tek kamerayla ölçek BİLİNEMEZ")
scale = np.linalg.norm(t) / np.linalg.norm(te)   # ölçeği dışarıdan (ör. bacak odometrisi) aldığımızı varsayalım
print("ölçek bacak odometrisinden alınınca 3B nokta hatası (medyan):",
      np.median(np.linalg.norm(Xs * scale - X, axis=1)).round(3), "m")
d = X[:, 2]; err = np.linalg.norm(Xs * scale - X, axis=1)
for lo, hi in [(2, 3), (3, 4), (4, 5)]:
    m = (d >= lo) & (d < hi); print(f"  derinlik {lo}-{hi} m: medyan hata {np.median(err[m]):.3f} m")
P1 = np.c_[np.eye(3), np.zeros(3)]; P2 = np.c_[R, t]                  # poz BİLİNSEYDİ (kalibre stereo gibi)
Xk = np.array([triangulate(P1, P2, ai, bi) for ai, bi in zip(n1, n2)]); ek = np.abs(Xk[:, 2] - X[:, 2])
m = (d >= 3.5) & (d < 4.5)
print("poz biliniyorsa, derinlik 3.5-4.5 m'de derinlik hatası RMS:",
np.sqrt((ek[m]**2).mean()).round(3), "m")
print("derinlik belirsizliği formülü  σ_z ≈ z² σ_px / (f b):  z =4 m, b =0.2 m, σ =0.5 px ->", round(16 * 0.5 / (500 * 0.2), 3), "m")
```

Çıktı:

```
epipolar kısıt  x2ᵀ E x1 (gürültüsüz) = 0.0e+00   (gürültülü ort. |.|: 2.6e-04)
öteleme yönü hatası (30 deneme medyanı, derece):
  gürültü 0.1 px:  ham 8-nokta   1.00°   Hartley normalizasyonlu   1.45°
```

```
  gürültü 0.5 px:  ham 8-nokta  39.42°   Hartley normalizasyonlu   6.40°
  gürültü 1.0 px:  ham 8-nokta  84.74°   Hartley normalizasyonlu   6.56°
4 adaydan seçilen: iki kameranın önünde kalan nokta sayısı 40 / 40
dönme hatası 0.561°;  öteleme yönü hatası 6.191°
|t| tahmini: 1.0  (gerçek 0.201) -> tek kamerayla ölçek BİLİNEMEZ
ölçek bacak odometrisinden alınınca 3B nokta hatası (medyan): 0.467 m
  derinlik 2-3 m: medyan hata 0.234 m
  derinlik 3-4 m: medyan hata 0.461 m
  derinlik 4-5 m: medyan hata 0.705 m
poz biliniyorsa, derinlik 3.5-4.5 m'de derinlik hatası RMS: 0.068 m
derinlik belirsizliği formülü  σ_z ≈ z² σ_px / (f b):  z =4 m, b =0.2 m, σ =0.5 px -> 0.08 m
```



Şekil 3.1: Üstten görünüş: gerçek noktalar ve üçgenlenmiş noktalar. Uzaktaki noktalar daha hatalı (derinlik hatası $\propto z^2$).

Bölüm 4

Nokta Bulutu Hizalama: Kabsch ve ICP

4.1 Kabsch (eşleşmeler biliniyorsa)

$\mathbf{q}_i \approx \mathbf{R}\mathbf{p}_i + \mathbf{t}$ en küçük kareler problemi kapalı formda çözülür:

1. Ağırlık merkezlerini çıkar: $\mathbf{p}'_i = \mathbf{p}_i - \bar{\mathbf{p}}$, $\mathbf{q}'_i = \mathbf{q}_i - \bar{\mathbf{q}}$.
2. $\mathbf{H} = \sum \mathbf{p}'_i \mathbf{q}'^T_i$, SVD: $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.
3. $\mathbf{R} = \mathbf{V}\,\mathrm{diag}(1, \dots, 1, \det(\mathbf{V}\mathbf{U}^T))\,\mathbf{U}^T$ (son eleman yansımayı engeller), $\mathbf{t} = \bar{\mathbf{q}} - \mathbf{R}\bar{\mathbf{p}}$.

Aynı formül 2B ve 3B'de çalışır**Türetme.** Amaç $\min_{\mathbf{R},\mathbf{t}} \sum \|\mathbf{q}_i - \mathbf{R}\mathbf{p}_i - \mathbf{t}\|^2$. (1) $\mathbf{t}$'ye göre türev sıfır: $\sum(\mathbf{q}_i - \mathbf{R}\mathbf{p}_i - \mathbf{t}) = 0 \Rightarrow \mathbf{t} = \bar{\mathbf{q}} - \mathbf{R}\bar{\mathbf{p}}$: merkezler hizalanır. (2) Yerine koyunca merkezlenmiş noktalarla $\sum \|\mathbf{q}'_i - \mathbf{R}\mathbf{p}'_i\|^2 = \sum(\|\mathbf{q}'_i\|^2 + \|\mathbf{p}'_i\|^2) - 2\sum \mathbf{q}'^T_i \mathbf{R}\mathbf{p}'_i$ (dönme boyu korur). İlk toplam sabit; en aza indirmek $\sum \mathbf{q}'^T_i \mathbf{R}\mathbf{p}'_i = \mathrm{iz}(\mathbf{R}\mathbf{H})$'yi ($\mathbf{H} = \sum \mathbf{p}'_i \mathbf{q}'^T_i$) en büyütmektir. (3) $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$: $\mathrm{iz}(\mathbf{R}\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T) = \mathrm{iz}(\underbrace{\mathbf{V}^T\mathbf{R}\mathbf{U}}_{\mathbf{W}}\,\mathbf{\Sigma}) = \sum_i W_{ii}\sigma_i$. $\mathbf{W}$ dik bir matris, köşegen elemanları en fazla 1; toplam en büyük $\mathbf{W} = \mathbf{I}$ iken: $\mathbf{R} = \mathbf{V}\mathbf{U}^T$. Bu bir yansıma çıkarsa ($\det = -1$), en küçük σ'ya karşılık gelen köşegeni -1 yapmak en az kayıplı düzeltmedir: formüldeki $\mathrm{diag}(1, \dots, \det)$. ; AprilTag köşelerinden, stereo noktalarından veya hareket yakalama işaretlerinden poz bulmanın çekirdeğidir.

4.2 ICP (eşleşmeler bilinmiyorsa)

Lidar veya derinlik kamerası iki taramada *aynı* noktaları ölçmez. **Iterative Closest Point:** (1) mevcut tahminle taramayı dönüştür, (2) her noktaya haritadaki en yakın noktayı eşle, (3) uzak eşleşmeleri at, (4) Kabsch ile adımı çöz, (5) tekrarla. Yerel bir yöntemdir: **iyi bir başlangıç tahmini gerekir** (odometri, IMU). Varyantlar: nokta-düzlem ICP (daha hızlı yakınsar), GICP, NDT; en yakın komşu için k-d ağacı.

ICP neden yakınsar (yerel olarak)? Her iterasyonda iki adım da toplam karesel mesafeyi artırmaz: en yakın komşuyu seçmek her eşleşmenin mesafesini (o anki dönüşümle) küçültür veya korur; Kabsch o eşleşmelerle mesafeyi en aza indirir. Azalan ve alttan sınırlı bir dizi yakınsar; ama vardığı yer yerel bir minimumdur.

Sayılar: 4×3 m oda + kolon, 1 cm gürültü

Gerçek hareket 8.59°, (0.30, −0.20) m. ICP 17 iterasyonda 8.57°, (0.2997, −0.1998); RMS eşleşme mesafesi 0.154 → 0.013 m (son değer, 1 cm gürültü + tarama seyrekliği). Başlangıç tahmini kötüleşince:

gerçek dönme	8.6°	25.8°	43.0°	65.9°	94.5°
açı hatası	0.07°	0.04°	0.05°	**49.5°**	**89.8°**

≈ 45°'nin ötesinde yerel minimuma düşüyor: dikdörtgen oda 90° simetrisine yakın; duvarlar

yanlış duvarlarla eşleşiyor. C++ sürümü de aynı davranışı gösterdi (65.9°'de 13.5°'de takıldı).

Python

```python
import numpy as np
rng = np.random.default_rng(4)

# Kabsch: eşleşmesi BİLİNEN iki nokta kümesi arasındaki en iyi R, t (SVD ile kapalı form)
def kabsch(P, Q):
    """Q ≈ R P + t en küçük kareler. P, Q: (N, d)."""
    cp, cq = P.mean(0), Q.mean(0)
    Hm = (P - cp).T @ (Q - cq)
    U, _, Vt = np.linalg.svd(Hm)
    D = np.eye(P.shape[1]); D[-1, -1] = np.sign(np.linalg.det(Vt.T @ U.T))   # yansımayı engelle
    R = Vt.T @ D @ U.T
    return R, cq - R @ cp

def rot2(a): c, s = np.cos(a), np.sin(a); return np.array([[c, -s], [s, c]])

# Elle kontrol: 3 nokta, 90° dönme + (1, 2) öteleme
P3 = np.array([[0, 0], [1, 0], [0, 1.0]]); Q3 = P3 @ rot2(np.pi / 2).T + [1, 2]
R3, t3 = kabsch(P3, Q3)
print("Kabsch 3 nokta: açı =", np.degrees(np.arctan2(R3[1, 0], R3[0, 0])).round(6), "°, t =",
t3.round(6))

# ICP: eşleşme BİLİNMİYOR. Bir oda taraması (lidar/derinlik kamerasından 2B kesit)
def room(n):
    s = rng.uniform(0, 4, n); pts = []
    for si in s:                                        # 4x3 m dikdörtgen oda + bir kolon
        if si < 1:   pts.append([si * 4, 0])
        elif si < 2: pts.append([4, (si - 1) * 3])
        elif si < 3: pts.append([(3 - si) * 4, 3])
        else:        pts.append([0, (4 - si) * 3])
    pts = np.array(pts)
    a = rng.uniform(0, 2 * np.pi, n // 4); col = np.c_[2.5 + 0.2 * np.cos(a), 1.2 + 0.2 * np.sin(a)]
    return np.r_[pts, col]

M = room(400)                                           # harita (referans tarama)
ang_true, t_true = 0.15, np.array([0.30, -0.20])        # robot hareket etti
S = (M[::2] - t_true) @ rot2(ang_true) + rng.normal(0, 0.01, (len(M[::2]), 2))   # yeni tarama, robot
çerçevesinde

def icp(S, M, R=np.eye(2), t=np.zeros(2), iters=40, reject=0.5):
    hist = []
    for k in range(iters):
        Sw = S @ R.T + t
        d2 = ((Sw[:, None, :] - M[None, :, :])**2).sum(2)      # kaba kuvvet en yakın komşu (k-d ağacı
        daha hızlı)
        j = d2.argmin(1); dist = np.sqrt(d2[np.arange(len(S)), j])
        ok = dist < reject                                     # uzak eşleşmeleri at
        dR, dt = kabsch(Sw[ok], M[j[ok]])
        R, t = dR @ R, dR @ t + dt
        hist.append(np.sqrt((dist[ok]**2).mean()))
        if k > 2 and abs(hist[-2] - hist[-1]) < 1e-6: break
    return R, t, hist

R, t, hist = icp(S, M)
a = np.arctan2(R[1, 0], R[0, 0])
```

```python
print(f"ICP: {len(hist)} iterasyon, açı {np.degrees(a):.3f}° (gerçek {np.degrees(ang_true):.3f}°), t = {t.round(4)} (gerçek {t_true})")
print("RMS eşleşme mesafesi: ilk", hist[0].round(4), "son", hist[-1].round(4), "m")

print("başlangıç tahmini kötüleşince (gerçek açı 0.15 rad'ın üzerine eklenen hata):")
for da in [0.0, 0.3, 0.6, 1.0, 1.5]:
    St = (M[::2] - t_true) @ rot2(ang_true + da) + rng.normal(0, 0.01, (len(M[::2]), 2))
    R, t, h = icp(St, M, reject=1.0)
    err = np.degrees(abs(np.arctan2(R[1, 0], R[0, 0]) - ang_true - da))
    print(f"  {np.degrees(ang_true + da):5.1f}° dönme: bulunan açının hatası {err:7.2f}°  -> {'YAKINSADI' if err < 1 else 'yerel minimum'}")
```

Çıktı:

```
Kabsch 3 nokta: açı = 90.0 °, t = [1. 2.]
ICP: 17 iterasyon, açı 8.572° (gerçek 8.594°), t = [ 0.2997 -0.1998] (gerçek [ 0.3 -0.2])
RMS eşleşme mesafesi: ilk 0.1543 son 0.0126 m
başlangıç tahmini kötüleşince (gerçek açı 0.15 rad'ın üzerine eklenen hata):
    8.6° dönme: bulunan açının hatası    0.07°  -> YAKINSADI
   25.8° dönme: bulunan açının hatası    0.04°  -> YAKINSADI
   43.0° dönme: bulunan açının hatası    0.05°  -> YAKINSADI
   65.9° dönme: bulunan açının hatası   49.49°  -> yerel minimum
   94.5° dönme: bulunan açının hatası   89.82°  -> yerel minimum
```

C++ (Eigen)

```cpp
// p4_icp.cpp — Kabsch (SVD) ve nokta-nokta ICP, Eigen ile. 2B oda taraması.
#include <Eigen/Dense>
#include <cmath>
#include <cstdio>
#include <random>
#include <vector>
using Mat2 = Eigen::Matrix2d;
using Vec2 = Eigen::Vector2d;
using Cloud = Eigen::Matrix<double, Eigen::Dynamic, 2>;   // her satır bir nokta

struct Pose2 { Mat2 R = Mat2::Identity(); Vec2 t = Vec2::Zero(); };

Pose2 kabsch(const Cloud& P, const Cloud& Q) {
    const Eigen::RowVector2d cp = P.colwise().mean(), cq = Q.colwise().mean();
    const Mat2 H = (P.rowwise() - cp).transpose() * (Q.rowwise() - cq);
    Eigen::JacobiSVD<Mat2> svd(H, Eigen::ComputeFullU | Eigen::ComputeFullV);
    Mat2 D = Mat2::Identity();
    D(1, 1) = (svd.matrixV() * svd.matrixU().transpose()).determinant() < 0 ? -1.0 : 1.0;   // yansıma yok
    Pose2 T;
    T.R = svd.matrixV() * D * svd.matrixU().transpose();
    T.t = cq.transpose() - T.R * cp.transpose();
    return T;
}

Mat2 rot(double a) { Mat2 R; R << std::cos(a), -std::sin(a), std::sin(a), std::cos(a); return R; }

Cloud room(int n, std::mt19937& g) {
    std::uniform_real_distribution<double> U(0, 4), A(0, 2 * M_PI);
    Cloud C(n + n / 4, 2);
    for (int i = 0; i < n; ++i) {
        double s = U(g);
        if (s < 1)      C.row(i) << s * 4, 0;
```

```
        else if (s < 2) C.row(i) << 4, (s - 1) * 3;
        else if (s < 3) C.row(i) << (3 - s) * 4, 3;
        else            C.row(i) << 0, (4 - s) * 3;
    }
    for (int i = 0; i < n / 4; ++i) { double a = A(g); C.row(n + i) << 2.5 + 0.2 * std::cos(a), 1.2 + 0.2 * std::sin(a); }
    return C;
}

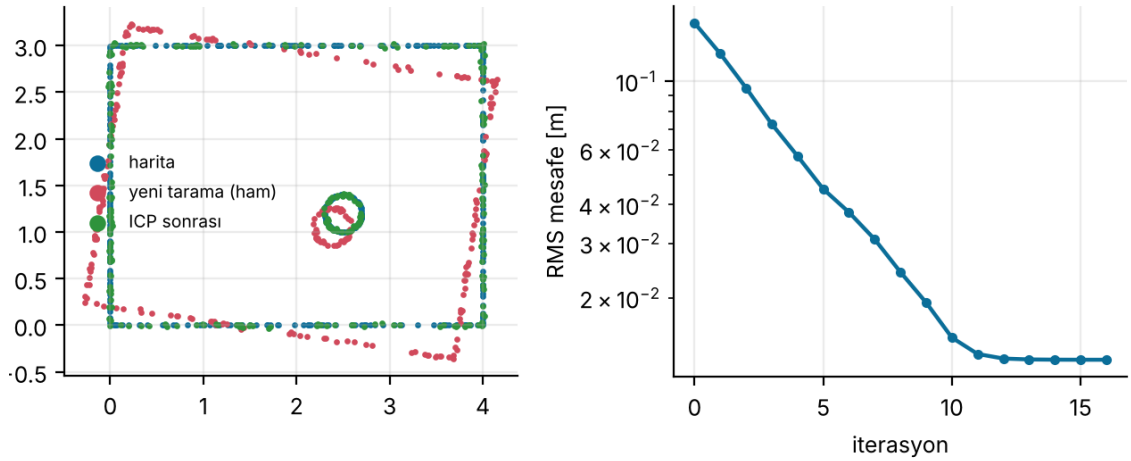
// ICP: her iterasyonda en yakın komşuyu bul, uzak eşleşmeleri at, Kabsch ile adımı çöz, birleştir
Pose2 icp(const Cloud& S, const Cloud& M, int& iters_used, double& rms, double reject = 0.5) {
    Pose2 T; double prev = 1e9;
    for (int it = 0; it < 40; ++it) {
        Cloud Sw = (S * T.R.transpose()).rowwise() + T.t.transpose();
        std::vector<int> src, dst; double sum = 0;
        for (int i = 0; i < Sw.rows(); ++i) {
            Eigen::Index j; double d2 = (M.rowwise() - Sw.row(i)).rowwise().squaredNorm().minCoeff(&j);
            if (d2 < reject * reject) { src.push_back(i); dst.push_back(int(j)); sum += d2; }
        }
        Cloud A(src.size(), 2), B(src.size(), 2);
        for (size_t k = 0; k < src.size(); ++k) { A.row(k) = Sw.row(src[k]); B.row(k) = M.row(dst[k]); }
        Pose2 d = kabsch(A, B);
        T.R = d.R * T.R; T.t = d.R * T.t + d.t;
        rms = std::sqrt(sum / src.size()); iters_used = it + 1;
        if (std::abs(prev - rms) < 1e-6) break;
        prev = rms;
    }
    return T;
}

int main() {
    Cloud P3(3, 2); P3 << 0, 0, 1, 0, 0, 1;
    Cloud Q3 = (P3 * rot(M_PI / 2).transpose()).rowwise() + Eigen::RowVector2d(1, 2);
    Pose2 T3 = kabsch(P3, Q3);
    std::printf("Kabsch 3 nokta: açı = %.4f°, t = (%.4f, %.4f)\n", std::atan2(T3.R(1, 0), T3.R(0, 0)) * 180 / M_PI, T3.t(0), T3.t(1));

    std::mt19937 g(4); std::normal_distribution<double> N(0, 0.01);
    Cloud M = room(400, g);
    const double ang = 0.15; const Vec2 tt(0.30, -0.20);
    for (double da : {0.0, 0.6, 1.0}) {
        Cloud S(M.rows() / 2, 2);
        for (int i = 0; i < S.rows(); ++i) S.row(i) = (rot(ang + da).transpose() * (M.row(2 * i).transpose() - tt)).transpose() + Eigen::RowVector2d(N(g), N(g));
        int it; double rms;
        Pose2 T = icp(S, M, it, rms, 1.0);
        double a = std::atan2(T.R(1, 0), T.R(0, 0));
        std::printf("gerçek dönme %5.1f°: ICP %2d iterasyon, açı %7.3f°, t = (%.4f, %.4f), RMS %.4f m\n",
                    (ang + da) * 180 / M_PI, it, a * 180 / M_PI, T.t(0), T.t(1), rms);
    }
    return 0;
}
```

Çıktı:

```
Kabsch 3 nokta: açı = 90.0000°, t = (1.0000, 2.0000)
gerçek dönme   8.6°: ICP 17 iterasyon, açı   8.549°, t = (0.2978, -0.1995), RMS 0.0121 m
gerçek dönme  43.0°: ICP 40 iterasyon, açı  42.909°, t = (0.2993, -0.1974), RMS 0.0126 m
gerçek dönme  65.9°: ICP 13 iterasyon, açı  13.532°, t = (0.5020, -0.3966), RMS 0.4795 m
```



Şekil 4.1: Sol: harita, ham yeni tarama ve ICP sonrası. Sağ: RMS eşleşme mesafesinin iterasyonlarla düşüşü.

C++ notu: Eigen'de satır işlemleri

`M.rowwise() - Sw.row(i)` her satırdan bir satır vektörünü çıkarır (NumPy'daki yayınlamanın, *broadcasting*, karşılığı); `.rowwise().squaredNorm().minCoeff(&j)` en küçük değeri ve indisini birlikte verir. `using Cloud = Eigen::Matrix<double, Eigen::Dynamic, 2>`: satır sayısı çalışma zamanında, sütun sayısı derleme zamanında belli; bu, boyut hatalarını derleyicinin yakalamasını sağlar.

Bölüm 5

EKF-SLAM

5.1 Problem

SLAM (eşzamanlı konum belirleme ve haritalama): robot hem kendi pozunu hem de işaretlerin (landmark) konumlarını, ikisini de bilmeden kestirir. Bu bir tavuk-yumurta problemi gibi görünür, ama olasılıksal olarak tek bir durum vektörüyle çözülür:

$$\mathbf{x} = [x,\ y,\ \theta,\ l_{1x},\ l_{1y},\ \ldots,\ l_{nx},\ l_{ny}]^T.$$

Tahmin: sadece robot kısmı hareket modeliyle ilerler; kovaryansın robot satırları $\mathbf{F}_x$ ile çarpılır. **Güncelleme:** işaret i'ye uzaklık-yön ölçümü $\mathbf{z} = (r,\ \phi)$; Jacobian $\mathbf{H}$ sadece robot ve i. işaret sütunlarında sıfırdan farklı. Ama Kalman kazancı *tüm* durumu günceller: bir işareti görmek, onunla *ilişkili* (korelasyonlu) tüm işaretlerin ve robotun tahminini düzeltir. SLAM'in sihri bu korelasyonlardadır.

EKF'nin iki adımının türetmesi. *Tahmin:* durum $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)+$ gürültü. Doğrusallaştır: $f(\mathbf{x}) \approx f(\hat{\mathbf{x}}) + \mathbf{F}(\mathbf{x} - \hat{\mathbf{x}})$, $\mathbf{F} = \partial f/\partial \mathbf{x}$. Hata $\mathbf{x} - \hat{\mathbf{x}}$'in kovaryansı $\mathbf{P}$ ise doğrusal dönüşümden sonra $\mathbf{F}\mathbf{P}\mathbf{F}^T$; giriş gürültüsü $\mathbf{Q}$ (odometri) $\mathbf{F}_u = \partial f/\partial \mathbf{u}$ üzerinden $\mathbf{F}_u\mathbf{Q}\mathbf{F}_u^T$ ekler. Hareket modeli $x' = x + v\Delta t\cos\theta$, $y' = y + v\Delta t\sin\theta$, $\theta' = \theta + \omega\Delta t$'nin türevleri kodda yazılı: $\partial x'/\partial\theta = -v\Delta t\sin\theta$ vb.

Ölçüm Jacobian'ı: $\Delta = (l_x - x,\ l_y - y)$, $q = \|\Delta\|^2$. Uzaklık $r = \sqrt{q}$: $\partial r/\partial l_x = \Delta_x/r$, $\partial r/\partial x = -\Delta_x/r$. Yön $\phi = \operatorname{atan2}(\Delta_y, \Delta_x) - \theta$: $\partial\phi/\partial l_x = -\Delta_y/q$, $\partial\phi/\partial l_y = \Delta_x/q$, $\partial\phi/\partial\theta = -1$ ($\frac{d}{dx}\operatorname{atan2}(y,x) = -y/(x^2+y^2)$).

Güncelleme: tek boyutta, önsel $(\hat{x}, P)$ ve ölçüm (z, R)'yi ağırlıklı birleştir: $\hat{x}^+ = \hat{x} + K(z - \hat{x})$. Sonsal varyans $(1-K)^2P + K^2R$; K'ya göre türevi sıfırla: $K = P/(P+R)$. Çok boyutlu karşılığı $\mathbf{K} = \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R})^{-1}$, $\mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}$. Kazanç, "hangisine ne kadar güvenmeli" sorusunun varyans oranıyla cevabı.

Yeni işaret: ilk görüşte konumu ölçümden ters çevirerek başlatılır: $\mathbf{l} = (x, y) + r(\cos(\phi+\theta), \sin(\phi+\theta))$; kovaryansı başta çok büyük ("bilinmiyor"), ilk güncelleme onu ölçüm belirsizliğine indirir.

Döngü kapatma: robot uzun süre yeni bölgelerde gezip belirsizliği büyüdükten sonra eski bir işareti tekrar görünce, belirsizlik bir anda çöker ve yol boyunca biriken hata geriye doğru düzelir.

EKF-SLAM'in sınırları

Kovaryans $(3+2n)^2$: 1000 işarette 4 milyon eleman, her güncelleme $O(n^2)$. Doğrusallaştırma hataları zamanla filtreyi *aşırı emin* (tutarsız) yapabilir. Yanlış eşleştirme (data association) tek başına haritayı bozar. Bu yüzden modern sistemler poz grafiği (Bölüm 6) veya kayan pencere optimizasyonu kullanır. EKF-SLAM yine de kavramları öğrenmek için en temiz yoldur.

Sayılar: ∼3 m yarıçaplı daire, işaretler sadece yarısında, menzil 1.6 m

Robot 321.–499. adımlar arasında hiçbir işaret görmüyor.

adım	ölü hesap hatası	EKF-SLAM hatası	EKF 1σ
100	0.092	0.054	0.064
300	0.310	0.135	0.222
400 (kör)	0.557	0.088	0.305
470 (kör)	0.587	0.067	**0.409**
520 (döngü kapandı)	0.579	0.080	**0.073**
600	0.396	0.045	0.032

Kör bölgede filtre belirsizliğinin büyüdüğünü dürüstçe söylüyor (1σ 0.41 m), işaretleri tekrar görünce 0.07 m'ye çöküyor. Bu koşuda kör bölgedeki gerçek hata (0.07 m) şanslı çıktı; önemli olan 1σ'nın davranışı. 8 işaretin hataları 0.6–15 cm. Son adımda NEES 3.85 (2 serbestlik için beklenen ortalama 2; tek değer bunun etrafında dağılır, tutarlılığı ancak birçok koşunun ortalaması söyler).

Python

```python
import numpy as np
rng = np.random.default_rng(5)

# 2B EKF-SLAM: durum = [x, y, θ, l1x, l1y, ..., lnx, lny]. Ölçüm: işaretlere uzaklık + yön (range-bearing).
# İşaret eşleştirmesi biliniyor (AprilTag gibi kimlikli işaretler).
# Yörünge (0, 3) merkezli ~3 m yarıçaplı bir daire. İşaretler dairenin sadece bir yarısında: öbür yarıda robot "kör".
ang = np.radians([-90, -60, -30, 0, 30, 60, 90, -75]); rad = np.array([2.2, 3.8, 2.2, 3.8, 2.2, 3.8, 2.2, 3.9])
L = np.c_[rad * np.cos(ang), 3 + rad * np.sin(ang)]
nL = len(L); dt = 0.1
sig_v, sig_w = 0.05, 0.08           # odometri gürültüsü (hız m/s, açısal hız rad/s; her adım)
sig_r, sig_b = 0.05, np.radians(2)  # ölçüm gürültüsü
R_max = 1.6                         # sensör menzili

def wrap(a): return (a + np.pi) % (2 * np.pi) - np.pi
def motion(x, v, w):
    return np.array([x[0] + v * dt * np.cos(x[2]), x[1] + v * dt * np.sin(x[2]), wrap(x[2] + w * dt)])

# Gerçek yörünge: yaklaşık bir daire, sonra ilk bölgeye geri dönüş (döngü kapatma)
T = 620; u = [(0.35, 0.115)] * T
xt = np.array([0.0, 0.0, 0.0]); truth = [xt]
for v, w in u: xt = motion(xt, v, w); truth.append(xt)
truth = np.array(truth)

n = 3 + 2 * nL
mu = np.zeros(n); P = np.zeros((n, n)); P[3:, 3:] = np.eye(2 * nL) * 1e6   # işaretler başta "bilinmiyor"
seen = np.zeros(nL, bool)
Q = np.diag([sig_v**2, sig_w**2]); Rm = np.diag([sig_r**2, sig_b**2])
dr = np.zeros(3); dr_hist = [dr.copy()]; mu_hist = [mu[:3].copy()]; Pxx = [0.0]; land_unc = []

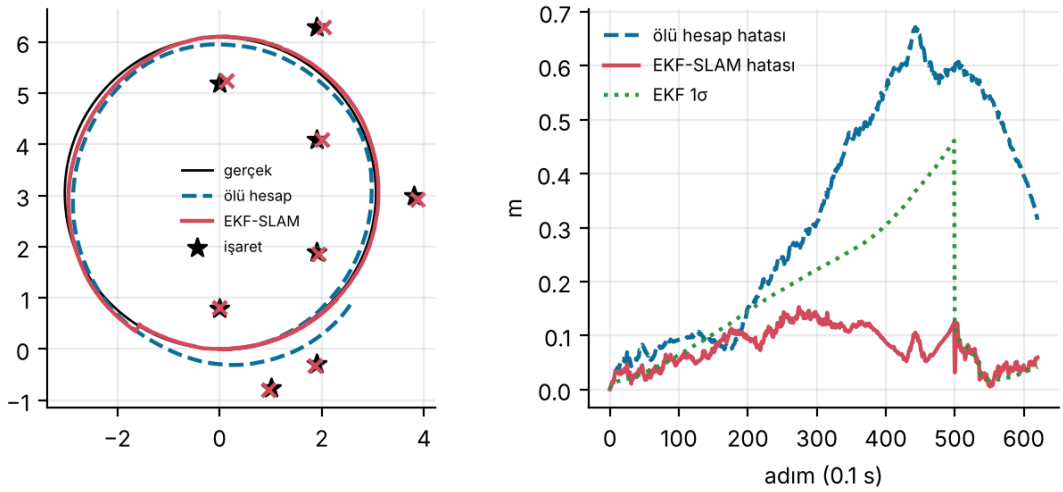
for k, (v, w) in enumerate(u):
    vn, wn = v + rng.normal(0, sig_v), w + rng.normal(0, sig_w)        # ölçülen odometri
    dr = motion(dr, vn, wn); dr_hist.append(dr.copy())
    # --- tahmin (predict): sadece robot durumu hareket eder
    th = mu[2]
    Fx = np.array([[1, 0, -vn * dt * np.sin(th)], [0, 1, vn * dt * np.cos(th)], [0, 0, 1]])
    Fu = np.array([[dt * np.cos(th), 0], [dt * np.sin(th), 0], [0, dt]])
    mu[:3] = motion(mu[:3], vn, wn)
```

```
    P[:3, :3] = Fx @ P[:3, :3] @ Fx.T + Fu @ Q @ Fu.T
    P[:3, 3:] = Fx @ P[:3, 3:]; P[3:, :3] = P[:3, 3:].T
    # --- güncelleme (update): menzildeki her işaret için
    x = truth[k + 1]
    for i in range(nL):
        d = L[i] - x[:2]; r = np.hypot(*d)
        if r > R_max: continue
        z = np.array([r + rng.normal(0, sig_r), wrap(np.arctan2(d[1], d[0]) - x[2] + rng.normal(0,
        sig_b))])
        j = 3 + 2 * i
        if not seen[i]:                                                    # ilk görüş: işareti başlat
            mu[j:j + 2] = mu[:2] + z[0] * np.array([np.cos(z[1] + mu[2]), np.sin(z[1] + mu[2])]);
            seen[i] = True
        dx = mu[j:j + 2] - mu[:2]; q = dx @ dx; sq = np.sqrt(q)
        zh = np.array([sq, wrap(np.arctan2(dx[1], dx[0]) - mu[2])])
        H = np.zeros((2, n))
        H[:, :3] = np.array([[-dx[0] / sq, -dx[1] / sq, 0], [dx[1] / q, -dx[0] / q, -1]])
        H[:, j:j + 2] = np.array([[dx[0] / sq, dx[1] / sq], [-dx[1] / q, dx[0] / q]])
        S = H @ P @ H.T + Rm; Kg = P @ H.T @ np.linalg.inv(S)
        inn = z - zh; inn[1] = wrap(inn[1])
        mu = mu + Kg @ inn; mu[2] = wrap(mu[2])
        P = (np.eye(n) - Kg @ H) @ P; P = 0.5 * (P + P.T)
    mu_hist.append(mu[:3].copy()); Pxx.append(np.sqrt(P[0, 0] + P[1, 1]))

dr_hist, mu_hist, Pxx = np.array(dr_hist), np.array(mu_hist), np.array(Pxx)
e_dr = np.linalg.norm(dr_hist[:, :2] - truth[:, :2], axis=1); e_sl = np.linalg.norm(mu_hist[:, :2] -
truth[:, :2], axis=1)
vis = np.zeros(T + 1, int)
for k in range(T + 1):
    vis[k] = (np.linalg.norm(L - truth[k, :2], axis=1) < R_max).sum()
print("işaret görmeyen adım aralığı:", np.where(vis == 0)[0].min(), "-", np.where(vis == 0)[0].max())
for k in [100, 200, 300, 400, 470, 520, 600]:
    print(f"adım {k:3d}: konum hatası  ölü hesap {e_dr[k]:5.3f} m   EKF-SLAM {e_sl[k]:5.3f} m   (EKF 1σ
    {Pxx[k]:.3f} m)")
le = np.linalg.norm(mu[3:].reshape(-1, 2) - L, axis=1)
print("işaret hataları [m]:", le[seen].round(3), " görülen:", seen.sum(), "/", nL)
print("durum boyutu", n, "-> kovaryans", n, "x", n, "; her güncelleme O(n²): 1000 işarette ~", (3 +
2000)**2 // 10**6, "milyon eleman")
# NEES tutarlılık kontrolü: hata, iddia edilen kovaryansla uyumlu mu? (2 serbestlik için ~2 beklenir)
print("son adımda konum için NEES:", float((mu[:2] - truth[-1, :2]) @ np.linalg.inv(P[:2, :2]) @
(mu[:2] - truth[-1, :2])).__round__(2))
```

Çıktı:

```
işaret görmeyen adım aralığı: 321 - 499
adım 100: konum hatası  ölü hesap 0.092 m   EKF-SLAM 0.054 m   (EKF 1σ 0.064 m)
adım 200: konum hatası  ölü hesap 0.151 m   EKF-SLAM 0.097 m   (EKF 1σ 0.145 m)
adım 300: konum hatası  ölü hesap 0.310 m   EKF-SLAM 0.135 m   (EKF 1σ 0.222 m)
adım 400: konum hatası  ölü hesap 0.557 m   EKF-SLAM 0.088 m   (EKF 1σ 0.305 m)
adım 470: konum hatası  ölü hesap 0.587 m   EKF-SLAM 0.067 m   (EKF 1σ 0.409 m)
adım 520: konum hatası  ölü hesap 0.579 m   EKF-SLAM 0.080 m   (EKF 1σ 0.073 m)
adım 600: konum hatası  ölü hesap 0.396 m   EKF-SLAM 0.045 m   (EKF 1σ 0.032 m)
işaret hataları [m]: [0.006 0.051 0.05  0.106 0.109 0.147 0.142 0.049]  görülen: 8 / 8
durum boyutu 19 -> kovaryans 19 x 19 ; her güncelleme O(n²): 1000 işarette ~ 4 milyon eleman
son adımda konum için NEES: 3.85
```



Şekil 5.1: Sol: gerçek yörünge, ölü hesap ve EKF-SLAM; yıldızlar gerçek, çarpılar kestirilen işaretler. Sağ: konum hataları ve EKF'nin kendi belirsizliği; döngü kapanınca belirsizlik çöküyor.

Bölüm 6

Poz Grafiği ve Gauss–Newton

6.1 Graf olarak SLAM

Filtre yerine *tüm geçmişi* bir optimizasyon problemi olarak kur: düğümler pozlar $\mathbf{x}_i = (x_i, y_i, \theta_i)$; kenarlar göreli ölçümler $\mathbf{z}_{ij}$ (odometri veya döngü kapatma) ve bilgi matrisleri $\mathbf{\Omega}_{ij} = \mathbf{\Sigma}_{ij}^{-1}$:

$$\min_{\mathbf{x}} \sum_{(i,j)} \mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}, \qquad \mathbf{e}_{ij} = \mathrm{t2v}\big(\mathbf{Z}_{ij}^{-1}\, \mathbf{X}_i^{-1} \mathbf{X}_j\big)\,.$$

Gauss–Newton: hatayı doğrusallaştır, $\mathbf{e}_{ij}(\mathbf{x}+\Delta\mathbf{x}) \approx \mathbf{e}_{ij} + \mathbf{A}_{ij}\Delta\mathbf{x}_i + \mathbf{B}_{ij}\Delta\mathbf{x}_j$; normal denklemler $\mathbf{H}\Delta\mathbf{x} = -\mathbf{b}$, $\mathbf{H} = \sum \mathbf{J}^T\mathbf{\Omega}\mathbf{J}$, $\mathbf{b} = \sum \mathbf{J}^T\mathbf{\Omega}\mathbf{e}$. Her kenar $\mathbf{H}$'ye sadece dört 3×3 blok ekler: $\mathbf{H}$ **seyrek**tir. İlk pozu sabitle (aksi halde tüm grafiği kaydırmak maliyeti değiştirmez: *gauge* özgürlüğü, $\mathbf{H}$ tekil).

Gauss–Newton'un türetmesi. Toplam maliyet $F(\mathbf{x}) = \sum \mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}$. Bir adım $\Delta\mathbf{x}$ için hataları birinci dereceden aç: $\mathbf{e}_{ij}(\mathbf{x}+\Delta\mathbf{x}) \approx \mathbf{e}_{ij} + \mathbf{J}_{ij}\Delta\mathbf{x}$ ($\mathbf{J}_{ij}$ sadece i ve j bloklarında sıfırdan farklı: $\mathbf{A}_{ij}$, $\mathbf{B}_{ij}$). Maliyet $\Delta\mathbf{x}$'te kuadratik olur:

$$F \approx \underbrace{\sum \mathbf{e}^T\mathbf{\Omega}\mathbf{e}}_{c} + 2\underbrace{\sum \mathbf{e}^T\mathbf{\Omega}\mathbf{J}}_{\mathbf{b}^T}\Delta\mathbf{x} + \Delta\mathbf{x}^T \underbrace{\sum \mathbf{J}^T\mathbf{\Omega}\mathbf{J}}_{\mathbf{H}}\Delta\mathbf{x}.$$

Türevi sıfırla: $\mathbf{H}\Delta\mathbf{x} = -\mathbf{b}$. Bu, Cilt 1'deki en küçük karelerin normal denklemleri; tek fark $\mathbf{\Omega}$ ağırlıkları ve her iterasyonda yeniden doğrusallaştırma.

Jacobian bloklarının türetmesi. $\mathbf{e} = \mathbf{R}_z^T(\mathbf{R}_i^T(\mathbf{t}_j - \mathbf{t}_i) - \mathbf{z}_t)$ ve açı hatası $\theta_j - \theta_i - z_\theta$. $\mathbf{t}_j$'ye göre türev $\mathbf{R}_z^T\mathbf{R}_i^T$, $\mathbf{t}_i$'ye göre eksisi; θ_i'ye göre $\mathbf{R}_z^T \frac{d\mathbf{R}_i^T}{d\theta_i}(\mathbf{t}_j - \mathbf{t}_i)$; açı satırında θ_j'ye $+1$, θ_i'ye -1. Kodda `A` ve `B` matrisleri bunlar.

Sağlık kontrolü: gürültü modeli doğruysa optimumda $\chi^2 \approx$ (ölçüm sayısı − bilinmeyen sayısı). Çok büyükse ya model yanlış ya da yanlış bir döngü kapatma var (robust kayıp fonksiyonları: Huber, Cauchy).

Bu, **bundle adjustment**'ın (kamera pozları + 3B noktalar, yeniden izdüşüm hatası) ve **faktör grafiklerinin** (GTSAM, Ceres, g2o) aynısıdır; sadece hata fonksiyonları farklı. Cilt 4B'deki trajectory optimization ile de aynı matematik: seyrek en küçük kareler.

Sayılar: 3 tur kare koridor, 121 poz

120 odometri + 27 döngü kapatma kenarı. χ^2: 8666 → 128 → 85.4 → 85.4 (3 iterasyonda yakınsama: Gauss–Newton'un karesel yakınsaması). Beklenen $441 - 360 = 81$: model tutarlı. Yörünge hatası (ATE RMS) 0.85 → 0.21 m; son poz hatası 1.16 → 0.04 m. $\mathbf{H}$ 363×363, sadece %2.4'ü sıfırdan farklı.
C++ (seyrek `SimplicialLDLT`): 3 tur ATE 0.63 → 0.09 m, 2.5 ms; **30 tur (1201 poz, 4710 kenar)** ATE 2.36 → 0.04 m, 7 iterasyon, 105 ms. Yoğun (dense) çözümle 3603×3603 matrisin her iterasyonda ayrıştırılması saniyeler sürerdi.

Python

```python
import numpy as np
rng = np.random.default_rng(6)

# 2B poz grafiği: düğüm = robot pozu (x, y, θ); kenar = iki poz arasında ölçülen göreli hareket.
# Odometri kenarları ardışık pozları bağlar; döngü kapatma kenarları "burayı daha önce gördüm" der.
def wrap(a): return (a + np.pi) % (2 * np.pi) - np.pi
def v2t(p):
    c, s = np.cos(p[2]), np.sin(p[2]); return np.array([[c, -s, p[0]], [s, c, p[1]], [0, 0, 1]])
def t2v(T): return np.array([T[0, 2], T[1, 2], np.arctan2(T[1, 0], T[0, 0])])
def between(a, b): return t2v(np.linalg.inv(v2t(a)) @ v2t(b))      # a'dan bakınca b

# Gerçek yörünge: 3 tur kare koridor (4 m x 4 m), her kenar 10 adım
truth = [np.zeros(3)]
for lap in range(3):
    for side in range(4):
        for k in range(10):
            p = truth[-1]; step = np.array([0.4, 0, 0]) if k < 9 else np.array([0.4, 0, np.pi / 2])
            truth.append(t2v(v2t(p) @ v2t(step)))
truth = np.array(truth); N = len(truth)

sig_o = np.array([0.02, 0.02, np.radians(1.5)])        # odometri kenarı gürültüsü
sig_l = np.array([0.05, 0.05, np.radians(2.0)])       # döngü kapatma (ör. tarama eşleştirme) gürültüsü
edges = []
for i in range(N - 1):
    edges.append((i, i + 1, between(truth[i], truth[i + 1]) + rng.normal(0, sig_o), np.diag(1 /
    sig_o**2)))
odo_edges = len(edges)
for i in range(N):                                    # aynı köşeye 40 adım sonra geri gelindi -> döngü
kenarı
    for j in range(i + 40, N, 40):
        if i % 5 == 0: edges.append((i, j, between(truth[i], truth[j]) + rng.normal(0, sig_l),
        np.diag(1 / sig_l**2)))
print("düğüm:", N, " odometri kenarı:", odo_edges, " döngü kapatma kenarı:", len(edges) - odo_edges)

# Başlangıç tahmini: odometriyi uç uca ekle (ölü hesap)
x0 = [np.zeros(3)]
for (i, j, z, _) in edges[:odo_edges]: x0.append(t2v(v2t(x0[-1]) @ v2t(z)))
x0 = np.array(x0)

def error_and_jac(xi, xj, z):
    Ri = v2t(xi)[:2, :2]; dt = xj[:2] - xi[:2]
    e = np.r_[Ri.T @ dt - z[:2], wrap(xj[2] - xi[2] - z[2])]
    # z'nin kendi dönmesini de hesaba katmak için hatayı z çerçevesine al
    Rz = v2t(z)[:2, :2]; e[:2] = Rz.T @ e[:2]
    dRiT = np.array([[-np.sin(xi[2]), np.cos(xi[2])], [-np.cos(xi[2]), -np.sin(xi[2])]])
    A = np.zeros((3, 3)); B = np.zeros((3, 3))
    A[:2, :2] = -Rz.T @ Ri.T; A[:2, 2] = Rz.T @ dRiT @ dt; A[2, 2] = -1
    B[:2, :2] = Rz.T @ Ri.T; B[2, 2] = 1
    return e, A, B

def chi2(x): return sum(e @ Om @ e for (i, j, z, Om) in edges for e in [error_and_jac(x[i], x[j],
z)[0]])

def gauss_newton(x, iters=10):
    x = x.copy(); hist = [chi2(x)]
    for it in range(iters):
        H = np.zeros((3 * N, 3 * N)); b = np.zeros(3 * N)
        for (i, j, z, Om) in edges:
            e, A, B = error_and_jac(x[i], x[j], z)
            si, sj = slice(3 * i, 3 * i + 3), slice(3 * j, 3 * j + 3)
```

```
            H[si, si] += A.T @ Om @ A; H[si, sj] += A.T @ Om @ B
            H[sj, si] += B.T @ Om @ A; H[sj, sj] += B.T @ Om @ B
            b[si] += A.T @ Om @ e; b[sj] += B.T @ Om @ e
        H[:3, :3] += np.eye(3) * 1e6                    # ilk pozu sabitle (gauge özgürlüğü)
        dx = np.linalg.solve(H, -b)
        x += dx.reshape(-1, 3); x[:, 2] = wrap(x[:, 2])
        hist.append(chi2(x))
        if np.abs(dx).max() < 1e-6: break
    return x, hist, H

def ate(x): return np.sqrt((np.linalg.norm(x[:, :2] - truth[:, :2], axis=1)**2).mean())
x, hist, H = gauss_newton(x0)
print("χ² iterasyonlarla:", [f"{h:.1f}" for h in hist])
print(f"beklenen χ² ≈ ölçüm - bilinmeyen = {3 * len(edges)} - {3 * (N - 1)} = {3 * len(edges) - 3 * (N - 1)} (gürültü modeli doğruysa)")
print(f"yörünge hatası (ATE RMS): ölü hesap {ate(x0):.3f} m  ->  optimize {ate(x):.3f} m")
print(f"son poz hatası: ölü hesap {np.linalg.norm(x0[-1, :2] - truth[-1, :2]):.3f} m -> {np.linalg.norm(x[-1, :2] - truth[-1, :2]):.3f} m")
print(f"H matrisi {H.shape[0]}x{H.shape[1]}, sıfır olmayan oranı {np.count_nonzero(H) / H.size * 100:.1f}%  (seyreklik: büyük grafiklerde seyrek Cholesky)")
```

Çıktı:

```
düğüm: 121  odometri kenarı: 120  döngü kapatma kenarı: 27
χ² iterasyonlarla: ['8666.1', '127.9', '85.4', '85.4', '85.4', '85.4', '85.4']
beklenen χ² ≈ ölçüm - bilinmeyen = 441 - 360 = 81 (gürültü modeli doğruysa)
yörünge hatası (ATE RMS): ölü hesap 0.851 m  ->  optimize 0.207 m
son poz hatası: ölü hesap 1.164 m -> 0.038 m
H matrisi 363x363, sıfır olmayan oranı 2.4%  (seyreklik: büyük grafiklerde seyrek Cholesky)
```

C++ (Eigen, seyrek)

```
// p6_posegraph.cpp — 2B poz grafiği, Gauss-Newton, SEYREK H ve Eigen::SimplicialLDLT
#include <Eigen/Dense>
#include <Eigen/Sparse>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <random>
#include <vector>
using Vec3 = Eigen::Vector3d;
using Mat3 = Eigen::Matrix3d;

double wrap(double a) { return std::remainder(a, 2 * M_PI); }
Mat3 v2t(const Vec3& p) { Mat3 T; T << std::cos(p[2]), -std::sin(p[2]), p[0], std::sin(p[2]), std::cos(p[2]), p[1], 0, 0, 1; return T; }
Vec3 t2v(const Mat3& T) { return {T(0, 2), T(1, 2), std::atan2(T(1, 0), T(0, 0))}; }
Vec3 between(const Vec3& a, const Vec3& b) { return t2v(v2t(a).inverse() * v2t(b)); }

struct Edge { int i, j; Vec3 z; Mat3 Om; };

// hata e = Z^{-1} (Xi^{-1} Xj) ve Jacobianlar A = de/dxi, B = de/dxj
void err_jac(const Vec3& xi, const Vec3& xj, const Vec3& z, Vec3& e, Mat3& A, Mat3& B) {
    Eigen::Matrix2d Ri = v2t(xi).topLeftCorner<2, 2>(), Rz = v2t(z).topLeftCorner<2, 2>();
    Eigen::Vector2d dt = xj.head<2>() - xi.head<2>();
    e.head<2>() = Rz.transpose() * (Ri.transpose() * dt - z.head<2>());
    e[2] = wrap(xj[2] - xi[2] - z[2]);
    Eigen::Matrix2d dRiT; dRiT << -std::sin(xi[2]), std::cos(xi[2]), -std::cos(xi[2]), -std::sin(xi[2]);
```

```
    A.setZero(); B.setZero();
    A.topLeftCorner<2, 2>() = -Rz.transpose() * Ri.transpose(); A.block<2, 1>(0, 2) = Rz.transpose() * dRiT * dt; A(2, 2) = -1;
    B.topLeftCorner<2, 2>() = Rz.transpose() * Ri.transpose(); B(2, 2) = 1;
}

int main() {
    std::mt19937 g(6); std::normal_distribution<double> N01(0, 1);
    // gerçek yörünge: kare koridor, `laps` tur
    for (int laps : {3, 30}) {
        std::vector<Vec3> truth{Vec3::Zero()};
        for (int l = 0; l < laps; ++l) for (int s = 0; s < 4; ++s) for (int k = 0; k < 10; ++k)
            truth.push_back(t2v(v2t(truth.back()) * v2t(Vec3(0.4, 0, k < 9 ? 0 : M_PI / 2))));
        const int n = truth.size();
        const Vec3 so(0.02, 0.02, 1.5 * M_PI / 180), sl(0.05, 0.05, 2.0 * M_PI / 180);
        auto noisy = [&](const Vec3& v, const Vec3& s) { return Vec3(v[0] + s[0] * N01(g), v[1] + s[1] * N01(g), v[2] + s[2] * N01(g)); };
        std::vector<Edge> E;
        for (int i = 0; i + 1 < n; ++i) E.push_back({i, i + 1, noisy(between(truth[i], truth[i + 1]), so), so.cwiseAbs2().cwiseInverse().asDiagonal()});
        const int nodo = E.size();
        for (int i = 0; i < n; i += 5) for (int j = i + 40; j < n; j += 40)
            E.push_back({i, j, noisy(between(truth[i], truth[j]), sl), sl.cwiseAbs2().cwiseInverse().asDiagonal()});

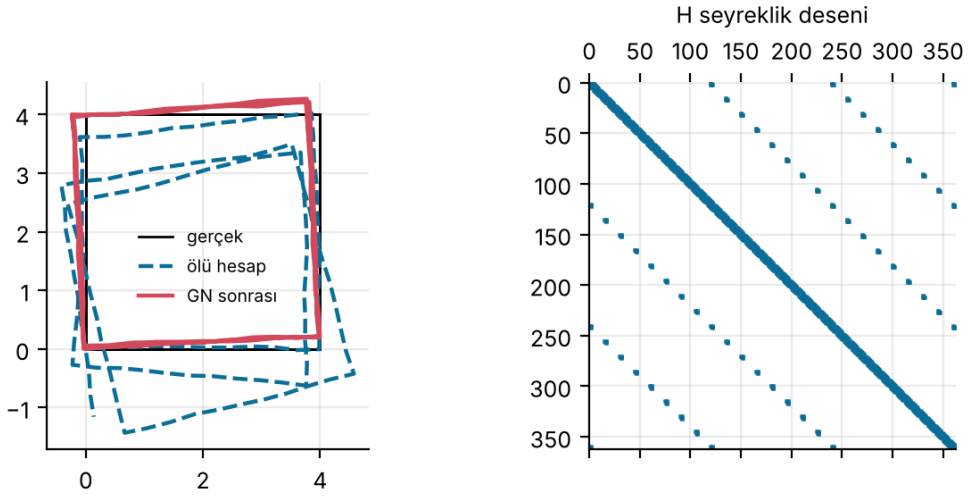
        std::vector<Vec3> x{Vec3::Zero()};                       // ölü hesap başlangıcı
        for (int k = 0; k < nodo; ++k) x.push_back(t2v(v2t(x.back()) * v2t(E[k].z)));
        auto ate = [&]() { double s = 0; for (int i = 0; i < n; ++i) s += (x[i].head<2>() - truth[i].head<2>()).squaredNorm(); return std::sqrt(s / n); };
        const double ate0 = ate();

        auto t0 = std::chrono::steady_clock::now();
        int it = 0; double chi2 = 0;
        for (; it < 10; ++it) {
            std::vector<Eigen::Triplet<double>> trip; Eigen::VectorXd b = Eigen::VectorXd::Zero(3 * n);
            chi2 = 0;
            for (const Edge& ed : E) {
                Vec3 e; Mat3 A, B; err_jac(x[ed.i], x[ed.j], ed.z, e, A, B);
                chi2 += e.dot(ed.Om * e);
                const Mat3 blocks[4] = {A.transpose() * ed.Om * A, A.transpose() * ed.Om * B, B.transpose() * ed.Om * A, B.transpose() * ed.Om * B};
                const int rows[4] = {ed.i, ed.i, ed.j, ed.j}, cols[4] = {ed.i, ed.j, ed.i, ed.j};
                for (int q = 0; q < 4; ++q) for (int r = 0; r < 3; ++r) for (int c = 0; c < 3; ++c)
                    trip.emplace_back(3 * rows[q] + r, 3 * cols[q] + c, blocks[q](r, c));   // aynı (r,c) toplanır
                b.segment<3>(3 * ed.i) += A.transpose() * ed.Om * e; b.segment<3>(3 * ed.j) += B.transpose() * ed.Om * e;
            }
            for (int r = 0; r < 3; ++r) trip.emplace_back(r, r, 1e6);                    // ilk pozu sabitle
            Eigen::SparseMatrix<double> H(3 * n, 3 * n); H.setFromTriplets(trip.begin(), trip.end());
            Eigen::SimplicialLDLT<Eigen::SparseMatrix<double>> ldlt(H);
            Eigen::VectorXd dx = ldlt.solve(-b);
            for (int i = 0; i < n; ++i) { x[i] += dx.segment<3>(3 * i); x[i][2] = wrap(x[i][2]); }
            if (dx.cwiseAbs().maxCoeff() < 1e-6) { ++it; break; }
        }
        double ms = std::chrono::duration<double, std::milli>(std::chrono::steady_clock::now() - t0).count();
        std::printf("%2d tur: %4d düğüm, %4zu kenar | ATE ölü hesap %.3f m -> GN %.3f m | %d iterasyon, son χ² %.1f (beklenen ~%zu) | %.1f ms\n",
```

```
                    laps, n, E.size(), ate0, ate(), it, chi2, 3 * E.size() - 3 * (n - 1), ms);
    }
    return 0;
}
```

Çıktı:

```
 3 tur:  121 düğüm,  147 kenar | ATE ölü hesap 0.630 m -> GN 0.086 m | 6 iterasyon, son χ² 66.7 (beklenen ~81) | 1.4
 ms
30 tur: 1201 düğüm, 4710 kenar | ATE ölü hesap 2.358 m -> GN 0.043 m | 7 iterasyon, son χ² 10411.7 (beklenen ~10530)
| 86.1 ms
```



Şekil 6.1: Sol: ölü hesap (kesik) ve Gauss–Newton sonrası yörünge. Sağ: **H**'nin seyreklik deseni; köşegen bant odometri, dağınık bloklar döngü kapatmalar.

C++ notu: `Triplet` ile seyrek matris kurmak

`std::vector<Eigen::Triplet<double>>` içine (satır, sütun, değer) üçlülerini sırasız ekle; `setFromTriplets` aynı konumdakileri *toplar*. Bu, "her kenar kendi bloklarını ekler" yapısına birebir uyar. `SimplicialLDLT` simetrik pozitif tanımlı seyrek sistemler için Cholesky ayrışımıdır; büyük problemlerde sıralama (AMD) dolgu (fill-in) miktarını belirler.

Bölüm 7

Bacaklı Robot Gövde Durum Kestirimi

Yürüyen bir robotun kontrolcüsü (Cilt 6: MPC, WBC) her adımda gövdenin *hızını* ve *yönelimini* ister. Gövde üzerinde iki iç (proprioceptive) kaynak vardır:

- **IMU:** ivme ve açısal hız, yüksek hızda (200–1000 Hz). Ama ivmeyi iki kez entegre etmek sapmayı (b) $\frac{1}{2}bt^2$ ile büyütür.
- **Bacak kinematiği:** yerdeki ayak kaymıyorsa, ayak hızı sıfırdır: $\dot{\mathbf{p}}_{\text{ayak}} = \mathbf{v}_{\text{gövde}} + \boldsymbol{\omega}\times\mathbf{r} + \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \;\Rightarrow\; \mathbf{v}_{\text{gövde}} = -\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} - \boldsymbol{\omega}\times\mathbf{r}$. Eklem enkoderleri zaten var; ek sensör gerekmez.

Kalman filtresi IMU'yu tahmin adımında, bacak hızını güncelleme adımında kullanır ve ivmeölçer sapmasını da kestirir. Temas, ya kuvvet sensöründen ya da kontrolcünün yürüme fazından ("şu an sol ayak yerde olmalı") bilinir.

Bacak hızı formülünün türetmesi. Ayak noktası dünyada $\mathbf{p}_f = \mathbf{p}_b + \mathbf{R}\mathbf{r}(\mathbf{q})$: gövde konumu + gövde çerçevesinde ayağın konumunun döndürülmüşü ($\mathbf{r}(\mathbf{q})$, Cilt 1'deki ileri kinematik). Zamana göre türev ($\dot{\mathbf{R}} = [\boldsymbol{\omega}]\mathbf{R}$, $\dot{\mathbf{r}} = \mathbf{J}\dot{\mathbf{q}}$):

$$\dot{\mathbf{p}}_f = \mathbf{v}_b + \boldsymbol{\omega}\times\mathbf{R}\mathbf{r} + \mathbf{R}\mathbf{J}(\mathbf{q})\dot{\mathbf{q}}.$$

Ayak kaymıyorsa $\dot{\mathbf{p}}_f = \mathbf{0} \Rightarrow \mathbf{v}_b = -\boldsymbol{\omega}\times\mathbf{R}\mathbf{r} - \mathbf{R}\mathbf{J}\dot{\mathbf{q}}$ (dünya çerçevesinde). Sağ taraftaki her şey ölçülüyor: $\boldsymbol{\omega}$ jiroskoptan, $\mathbf{q}, \dot{\mathbf{q}}$ enkoderlerden, $\mathbf{R}$ yönelim kestiriminden.

Kestirim modeli. Durum (p, v, b), ivmeölçer $a_{\text{ölç}} = a + b$+gürültü. Gerçek ivme $a = a_{\text{ölç}} - b$; sabit ivmeli kinematik: $p' = p + v\Delta t + \frac{1}{2}(a_{\text{ölç}} - b)\Delta t^2$, $v' = v + (a_{\text{ölç}} - b)\Delta t$, $b' = b$. Matris biçimi koddaki $\mathbf{F}$ ve $\mathbf{G}$. Ölçüm $z = v$ ($\mathbf{H} = [0\ 1\ 0]$). Sapma b, hız ölçümleriyle ivme tahmini arasındaki tutarsızlıktan *kendiliğinden* kestirilir: gözlemlenebilir.

Kapının istatistiği. Model doğruysa yenilik ν sıfır ortalamalı, kovaryansı $\mathbf{S} = \mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R}$ olan Gauss dağılımlıdır. $d^2 = \nu^T\mathbf{S}^{-1}\nu$ ölçüm boyutu kadar serbestlikli χ^2 dağılır; 1 boyutta $d^2 > 9$ olasılığı %0.27 (3σ). Daha büyükse "bu ölçüm modelimle tutarsız" deriz.

Kayma ve kapı: ayak kayarsa "ayak hızı sıfır" varsayımı çöker. Yenilik (innovation) $\nu = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}$ ve kovaryansı $\mathbf{S}$ ile **Mahalanobis uzaklığı** $d^2 = \nu^T\mathbf{S}^{-1}\nu$ hesaplanır; d^2, χ^2 dağılımının yüksek bir yüzdeliğini aşarsa ölçüm reddedilir (1 boyut için $9 = 3\sigma$).

Gözlemlenemezlik: yalnız IMU + bacakla mutlak konum (x, y, z) ve yön açısı (yaw) gözlemlenemez; hata zamanla sınırsız büyür. Yuvarlanma ve yunuslama (roll, pitch) yerçekimi sayesinde gözlemlenebilir. Konumu bağlamak için kamera/lidar (Bölüm 3–6) gerekir.

Sayılar: 20 s yürüme, 0.3 m/s, IMU sapması 0.15 m/s^2, 8–9.5 s arası ayak kayması

Yöntem	20 s sonra konum hatası	hız RMS hatası
Sadece IMU (çift integral)	23.8 m	1.47 m/s
Sadece bacak odometrisi	0.51 m	–
KF (IMU + bacak)	0.53 m	0.078 m/s
KF + 3σ kayma kapısı	0.013 m	0.012 m/s

Sapma 0.1494 olarak kestirildi (gerçek 0.15). Kayma sırasında kapısız KF'nin hız hatası 0.21 m/s; kapılı 0.002 m/s (600 ölçümün 583'ü reddedildi). Kapılı KF'nin 1.3 cm'lik konum hatası bu koşuda şanslı; konum gözlemlenemediği için başka tohumlarla birkaç cm'den onlarca cm'ye kadar değişir. C++ şablonlu KF: kapısız 0.53 m / 0.079 m/s, kapılı 0.007 m / 0.012 m/s, 579 ret.

Kapı iki ucu keskin bir bıçak

Kestirim bir kez gerçekten saparsa, doğru ölçümler de "tutarsız" görünüp sürekli reddedilebilir ve filtre bir daha toparlanmaz. Gerçek sistemlerde art arda çok ret olursa kapı gevşetilir veya filtre yeniden başlatılır.

Python

```python
import numpy as np
rng = np.random.default_rng(7)

# Bacaklı robot gövde durum kestirimi (yatay eksen, sadeleştirilmiş):
#   durum x = [p, v, b]  (konum, hız, ivmeölçer sapması)
#   giriş:  ivmeölçer a_ölç = a + b + gürültü  (400 Hz)
#   ölçüm:  ayak yerdeyken bacak kinematiğinden gövde hızı  v_bacak = -J(q) q̇  (+ gürültü)
dt = 0.0025; T = 20.0; n = int(T / dt); t = np.arange(n) * dt
v_true = 0.3 + 0.08 * np.sin(2 * np.pi * 1.5 * t)          # adım frekansında dalgalanan yürüme hızı
a_true = np.gradient(v_true, dt); p_true = np.cumsum(v_true) * dt
b_true = 0.15                                               # m/s² ivmeölçer sapması (ucuz IMU için tipik büyüklük)
a_meas = a_true + b_true + rng.normal(0, 0.05, n)
v_leg = v_true + rng.normal(0, 0.04, n)                     # bacak odometrisi hızı
slip = (t > 8) & (t < 9.5)                                  # ayak kayıyor: kinematik "yanlış" hız söylüyor
v_leg[slip] += 0.35
contact = np.ones(n, bool); contact[(t % (1 / 1.5)) < 0.03] = False   # adım değişiminde kısa çift-havada belirsizlik

F = np.array([[1, dt, -0.5 * dt**2], [0, 1, -dt], [0, 0, 1]]); G = np.array([0.5 * dt**2, dt, 0])
Q = np.diag([1e-8, (0.05 * dt)**2, (1e-3 * dt)**2]); Hm = np.array([[0, 1.0, 0]]); Rv = 0.04**2

def run(use_leg=True, gate=None):
    x = np.zeros(3); P = np.diag([1e-4, 1e-2, 0.1]); out = np.zeros((n, 3)); rej = 0
    for k in range(n):
        x = F @ x + G * a_meas[k]; P = F @ P @ F.T + Q          # IMU ile tahmin
        if use_leg and contact[k]:
            S = Hm @ P @ Hm.T + Rv; inn = v_leg[k] - x[1]
            if gate is not None and inn**2 / S[0, 0] > gate:     # Mahalanobis kapısı: tutarsız ölçümü reddet
                rej += 1; out[k] = x; continue
            Kg = (P @ Hm.T / S).ravel(); x = x + Kg * inn; P = (np.eye(3) - np.outer(Kg, Hm)) @ P
        out[k] = x
    return out, rej

imu, _ = run(use_leg=False)
vl = v_leg.copy()
for k in range(1, n):
    if not contact[k]: vl[k] = vl[k - 1]                    # temas yoksa son ölçülen hızı tut
leg_only = np.cumsum(vl) * dt
kf, _ = run()
kfg, rej = run(gate=9.0)                                     # 3σ kapı
def rep(name, pe, ve=None):
```

```
    s = f"{name:28s} 20 s sonra konum hatası {abs(pe[-1] - p_true[-1]):6.3f} m"
    if ve is not None: s += f",  hız RMS hatası {np.sqrt(((ve - v_true)**2).mean()):.3f} m/s"
    print(s)
rep("sadece IMU (çift integral)", imu[:, 0], imu[:, 1])
rep("sadece bacak odometrisi", leg_only)
rep("KF (IMU + bacak)", kf[:, 0], kf[:, 1])
rep("KF + kayma kapısı (3σ)", kfg[:, 0], kfg[:, 1])
print("kestirilen ivmeölçer sapması:", kf[-1, 2].round(4), "(gerçek", b_true, ")")
m = slip
print(f"kayma sırasında hız hatası (ort.): KF {np.abs(kf[m, 1] - v_true[m]).mean():.3f}  kapılı KF
{np.abs(kfg[m, 1] - v_true[m]).mean():.3f} m/s;  reddedilen ölçüm {rej}")
print("Not: konumu düzelten bir ölçüm yok -> konum hatası zamanla büyür (gözlemlenemez); kamera/lidar
gerekir.")
```

Çıktı:

```
sadece IMU (çift integral)   20 s sonra konum hatası 23.792 m,  hız RMS hatası 1.470 m/s
sadece bacak odometrisi      20 s sonra konum hatası  0.514 m
KF (IMU + bacak)             20 s sonra konum hatası  0.526 m,  hız RMS hatası 0.078 m/s
KF + kayma kapısı (3σ)       20 s sonra konum hatası  0.013 m,  hız RMS hatası 0.012 m/s
kestirilen ivmeölçer sapması: 0.1494 (gerçek 0.15 )
kayma sırasında hız hatası (ort.): KF 0.208  kapılı KF 0.002 m/s;  reddedilen ölçüm 583
Not: konumu düzelten bir ölçüm yok -> konum hatası zamanla büyür (gözlemlenemez); kamera/lidar gerekir.
```

C++: şablonlu Kalman filtresi sınıfı

```
// p7_kf.cpp — boyutları derleme zamanında sabit, şablonlu (template) Kalman filtresi
// + bacaklı gövde hız kestirimi (IMU tahmini, bacak kinematiği ölçümü, Mahalanobis kapısı)
#include <Eigen/Dense>
#include <cmath>
#include <cstdio>
#include <random>

template <int NX, int NU, int NZ>
class KalmanFilter {
public:
    using VecX = Eigen::Matrix<double, NX, 1>;
    using VecU = Eigen::Matrix<double, NU, 1>;
    using VecZ = Eigen::Matrix<double, NZ, 1>;
    using MatX = Eigen::Matrix<double, NX, NX>;
    using MatXU = Eigen::Matrix<double, NX, NU>;
    using MatZX = Eigen::Matrix<double, NZ, NX>;
    using MatZ = Eigen::Matrix<double, NZ, NZ>;

    KalmanFilter(const MatX& F, const MatXU& G, const MatX& Q) : F_(F), G_(G), Q_(Q) {}
    void init(const VecX& x0, const MatX& P0) { x_ = x0; P_ = P0; }

    void predict(const VecU& u) { x_ = F_ * x_ + G_ * u; P_ = F_ * P_ * F_.transpose() + Q_; }

    // gate > 0 ise: Mahalanobis² > gate olan ölçümü reddet; kabul edildiyse true döner
    bool update(const VecZ& z, const MatZX& H, const MatZ& R, double gate = -1) {
        const VecZ inn = z - H * x_;
        const MatZ S = H * P_ * H.transpose() + R;
        const double d2 = inn.dot(S.ldlt().solve(inn));
        if (gate > 0 && d2 > gate) return false;
        const Eigen::Matrix<double, NX, NZ> K = P_ * H.transpose() * S.inverse();
        x_ += K * inn;
        const MatX IKH = MatX::Identity() - K * H;
        P_ = IKH * P_ * IKH.transpose() + K * R * K.transpose();   // Joseph biçimi: sayısal olarak
simetrik ve pozitif kalır
```

```
        return true;
    }
    const VecX& x() const { return x_; }
    const MatX& P() const { return P_; }

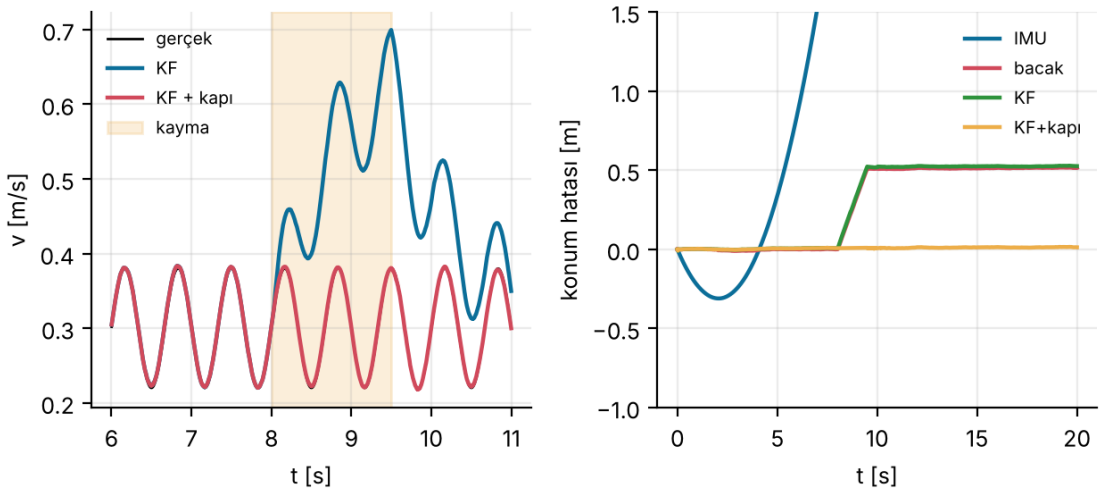
private:
    MatX F_; MatXU G_; MatX Q_;
    VecX x_ = VecX::Zero(); MatX P_ = MatX::Identity();
};

int main() {
    const double dt = 0.0025, T = 20.0, b_true = 0.15; const int n = int(T / dt);
    std::mt19937 g(7); std::normal_distribution<double> na(0, 0.05), nv(0, 0.04);
    Eigen::Matrix3d F; F << 1, dt, -0.5 * dt * dt, 0, 1, -dt, 0, 0, 1;
    Eigen::Vector3d G(0.5 * dt * dt, dt, 0);
    Eigen::Matrix3d Q = Eigen::Vector3d(1e-8, std::pow(0.05 * dt, 2), std::pow(1e-3 * dt,
2)).asDiagonal();
    Eigen::RowVector3d H(0, 1, 0);
    Eigen::Matrix<double, 1, 1> R; R << 0.04 * 0.04;

    for (double gate : {-1.0, 9.0}) {
        KalmanFilter<3, 1, 1> kf(F, G, Q);
        kf.init(Eigen::Vector3d::Zero(), Eigen::Vector3d(1e-4, 1e-2, 0.1).asDiagonal());
        g.seed(7);
        double p = 0, se = 0, slip_err = 0; int rej = 0, nslip = 0;
        for (int k = 0; k < n; ++k) {
            const double t = k * dt, v = 0.3 + 0.08 * std::sin(2 * M_PI * 1.5 * t);
            const double a = 0.08 * 2 * M_PI * 1.5 * std::cos(2 * M_PI * 1.5 * t);
            p += v * dt;
            const bool slip = t > 8 && t < 9.5, contact = std::fmod(t, 1 / 1.5) >= 0.03;
            Eigen::Matrix<double, 1, 1> u; u << a + b_true + na(g);
            kf.predict(u);
            Eigen::Matrix<double, 1, 1> z; z << v + nv(g) + (slip ? 0.35 : 0.0);
            if (contact && !kf.update(z, H, R, gate)) ++rej;
            se += std::pow(kf.x()[1] - v, 2);
            if (slip) { slip_err += std::abs(kf.x()[1] - v); ++nslip; }
            if (k == n - 1)
                std::printf("%s: konum hatası %.3f m, hız RMS %.3f m/s, kayma sırasında hız hatası %.3f
m/s, sapma %.4f, reddedilen %d\n",
                            gate > 0 ? "KF + 3σ kapı" : "KF (kapısız)", std::abs(kf.x()[0] - p),
std::sqrt(se / n), slip_err / nslip, kf.x()[2], rej);
        }
    }
    return 0;
}
```

Çıktı:

```
KF (kapısız): konum hatası 0.527 m, hız RMS 0.079 m/s, kayma sırasında hız hatası 0.209 m/s, sapma 0.1499,
reddedilen 0
KF + 3σ kapı: konum hatası 0.007 m, hız RMS 0.012 m/s, kayma sırasında hız hatası 0.001 m/s, sapma 0.1498,
reddedilen 579
```



Şekil 7.1: Sol: kayma aralığında (sarı) gövde hızı; kapısız KF kayan ayağa inanıyor. Sağ: konum hataları; IMU tek başına dakikalar içinde işe yaramaz hale geliyor.

C++ notu: şablon parametresi olarak boyut

`template <int NX, int NU, int NZ> class KalmanFilter`: durum, giriş ve ölçüm boyutları derleme zamanında sabit. Eigen bu durumda matrisleri yığında (stack) tutar; döngüde bellek ayırma (`malloc`) olmaz. Bu, 1 kHz gerçek zamanlı döngülerde önemlidir (Cilt 2). Kovaryans güncellemesinde **Joseph biçimi** $(\mathbf{I}-\mathbf{K}\mathbf{H})\mathbf{P}(\mathbf{I}-\mathbf{K}\mathbf{H})^{T}+\mathbf{K}\mathbf{R}\mathbf{K}^{T}$, kısa biçim $(\mathbf{I}-\mathbf{K}\mathbf{H})\mathbf{P}$'den daha pahalıdır ama yuvarlama hatalarıyla $\mathbf{P}$'nin simetrikliğini ve pozitifliğini bozmaz.

Bölüm 8

Gerçek Sistemler Haritası ve SB-1 için Algı

8.1 Bu cilttekiler gerçek sistemlerde nerede?

Sistem / araç	Ne yapar	Bu ciltte karşılığı
OpenCV	kalibrasyon, özellikler, homografi, **E**, PnP, RANSAC	Bölüm 1–3
AprilTag	basılı işaret: kimlik + 6B poz	Bölüm 1–2 (homografi), 4 (Kabsch)
ORB-SLAM3	görsel ve görsel-ataletsel SLAM	3 (epipolar), 6 (BA, poz grafiği)
VINS-Mono/Fusion, OpenVINS	görsel-ataletsel odometri	3, 5, 7
Cartographer, KISS-ICP, FAST-LIO2	lidar SLAM / odometri	4 (ICP), 6
GTSAM, Ceres, g2o	faktör grafiği / en küçük kareler çözücüleri	6
`robot_localization` (ROS 2)	EKF/UKF sensör füzyonu	5, 7
Pronto, Cerberus, kontak destekli InEKF	bacaklı robot durum kestirimi	7

8.2 Modern eğilimler (kavram düzeyinde)

- **Öğrenilmiş özellikler ve eşleştiriciler** (SuperPoint, LightGlue gibi): ORB/SIFT'in yerine sinir ağı; geometri (RANSAC, BA) aynı kalır.
- **Değişmez (invariant) EKF:** durumu Lie grubunda tanımlar (Cilt 1 Bölüm 3); bacaklı robotlarda tutarlılığı belirgin biçimde artırır.
- **Derinlik kameraları ve yükseklik haritaları:** bacaklı robotlar için "nereye basabilirim" sorusu (elevation mapping) SLAM kadar önemlidir.
- **Uçtan uca öğrenme:** pekiştirmeli öğrenme ile yürüyen robotlar (Cilt 6) çoğu zaman açık bir harita kurmaz; ama gövde durum kestiricisi yine kullanılır.

8.3 SB-1 için gerçekçi bir algı yolu (bütçesiz)

1. **Telefon kamerası kalibrasyonu:** kâğıda basılmış veya ekranda gösterilen dama tahtası, OpenCV `calibrateCamera`. Yeniden izdüşüm hatasını raporla.
2. **AprilTag pozu:** basılı etiket, `pupil-apriltags` veya OpenCV `aruco`. Etiketi cetvelle hareket ettirip ölçülen mesafeyle karşılaştır (Bölüm 3'teki z^2 yasasını doğrula).
3. **IMU sapması:** elinde bir IMU (MPU6050 sınıfı, ucuz) olursa: 60 s sabit kayıt, Allan varyansı ve Bölüm 7'deki $\frac{1}{2}bt^2$ sapmasını kendin gör.

4. **Simülasyonda SB-1:** MuJoCo'da eklem açıları + gövde IMU'su; Bölüm 7'deki KF'yi gerçek kinematikle ($\mathbf{v} = -\mathbf{J}\dot{\mathbf{q}}$) kur; ayağı kaydırıp kapıyı test et.

8.4 Kaynaklar

- Hartley & Zisserman, *Multiple View Geometry in Computer Vision* (Bölüm 1–3'ün klasik kaynağı).
- Szeliski, *Computer Vision: Algorithms and Applications*, 2. baskı (yazarın sitesinde ücretsiz).
- Barfoot, *State Estimation for Robotics*, 2. baskı (yazarın sitesinde ücretsiz PDF; Lie grupları ile kestirim).
- Thrun, Burgard, Fox, *Probabilistic Robotics* (EKF-SLAM, parçacık filtresi, GraphSLAM).
- Grisetti ve ark., "A Tutorial on Graph-Based SLAM" (Bölüm 6'nın doğrudan kaynağı).
- Bloesch ve ark., "State Estimation for Legged Robots: Consistent Fusion of Leg Kinematics and IMU" (RSS 2012).
- Hartley (R.) ve ark., "Contact-Aided Invariant Extended Kalman Filtering for Robot State Estimation" (IJRR 2020).
- Cyrill Stachniss'in YouTube'daki SLAM ve fotogrametri ders videoları (telefonda izlenebilir, staj dönemi için ideal).

Bölüm 9

Sorular

Kâğıt, kalem, hesap makinesi. **(K)**: kod okuma.

P1 — Kamera

P1.1 $f_x = f_y = 500$, $c = (320,\ 240)$. Kamera çerçevesindeki $(0.2,\ -0.1,\ 2.0)$ m noktasının pikseli? Aynı ışın üzerindeki $(0.4,\ -0.2,\ 4.0)$ noktasınınki?

P1.2 Aynı kamerada 1 piksel, 2 m ve 5 m uzaklıkta kaç milimetreye karşılık gelir? SB-1'in kamerası 1.5 m'deki 2 cm'lik bir engeli kaç piksel genişliğinde görür?

P1.3 $\mathbf{P} = \mathbf{K}[\mathbf{R}\ \mathbf{t}]$ neden 11 serbestliğe sahip (12 eleman olduğu halde)? DLT için en az kaç 3B–2B eşleşme gerekir?

P1.4 $k_1 = -0.2$, normalize koordinat $(0.25,\ 0.2)$. Bozulmuş normalize koordinat? Piksel cinsinden kayma ($f = 500$)?

P2 — Homografi ve RANSAC

P2.1 Homografinin serbestlik sayısı ve gereken en az nokta sayısı? Kamera sadece dönüyorsa neden sahne düzlemsel olmasa da homografi geçerlidir?

P2.2 İç oran %50, örnek boyutu 4 ve 8 için %99 başarıya kaç RANSAC iterasyonu gerekir?

P2.3 Hartley normalizasyonu ne yapar, neden gerekir?

P3 — Epipolar geometri

P3.1 **E**, **F** ve **H**'nin serbestlik sayıları? **E**'nin tekil değerleri nasıldır?

P3.2 Tek kamerayla ölçek neden bulunamaz? SB-1'de ölçeği nereden alabilirsin (iki yol)?

P3.3 Stereo kamera: $f = 500$ px, taban $b = 0.1$ m. Piksel farkı 10 px ise derinlik? Piksel farkı ölçüm hatası 0.5 px ise derinlik belirsizliği? Aynı hesap 2 m'de?

P4 — Kabsch ve ICP

P4.1 $\mathbf{p}_1 = (0,0)$, $\mathbf{p}_2 = (2,0)$; $\mathbf{q}_1 = (1,1)$, $\mathbf{q}_2 = (1,3)$. **R** ve **t**'yi elle bul, sağlamasını yap.

P4.2 ICP neden iyi bir başlangıç tahmini ister? Dikdörtgen bir odada neden $\approx 45°$'den sonra başarısız oldu?

P4.3 **(K)** `icp` fonksiyonundaki `reject` eşiği ne işe yarar? Çok küçük ve çok büyük seçmenin sonuçları?

P5 — EKF-SLAM

P5.1 2B EKF-SLAM'de 50 işaret: durum boyutu ve kovaryanstaki eleman sayısı?

P5.2 Tek boyutlu KF: önsel $\hat{x} = 10$, $P = 4$; ölçüm $z = 12$, $R = 1$. Kalman kazancı, sonsal kestirim ve varyans?

P5.3 Döngü kapatıldığında neden robotun *geçmiş* yörüngesi ve uzaktaki işaretler de düzelir?

P6 — Poz grafiği

P6.1 121 poz, 147 kenar (her biri 3 boyutlu ölçüm), ilk poz sabit. Optimumda beklenen χ^2? Gerçekte 2000 çıksaydı ne düşünürdün?

P6.2 İlk pozu sabitlemezsek $\mathbf{H}$ neden tekil olur?

P7 — Bacaklı kestirim

P7.1 İvmeölçer sapması 0.15 m/s^2, başka hata yok. Çift integral ile 10 s ve 20 s sonra konum hatası? Simülasyonda 20 s sonra neden 30 m değil 23.8 m çıktı?

P7.2 Tek eklemli, 0.3 m'lik dik bacak, ayak yerde sabit, kalça açısal hızı 1 rad/s. Gövdenin yatay hızı?

P7.3 Bacak hızı ölçüm gürültüsü $\sigma = 0.04$ m/s, filtrenin hız varyansı 10^{-4}. Ayak kayıyor ve ölçüm 0.35 m/s sapıyor. Mahalanobis d^2? 3σ kapısı ($d^2 > 9$) ölçümü reddeder mi?

Bölüm 10

Cevaplar

P1

P1.1 $u = 320 + 500 \cdot 0.2/2 = 370$, $v = 240 + 500 \cdot (-0.1)/2 = 215$. İkinci nokta aynı ışında: yine $(370,\ 215)$. Derinlik görüntüde kaybolur.

P1.2 $2/500 = 4$ mm, $5/500 = 10$ mm. 1.5 m'de piksel 3 mm: $20/3 \approx 6.7$ piksel.

P1.3 Homojen: $\mathbf{P}$ ve $\lambda\mathbf{P}$ aynı izdüşümü verir, ölçek serbest $\Rightarrow 12-1 = 11$. Her eşleşme 2 denklem: en az 6 (5.5'in yukarı yuvarlanması).

P1.4 $r^2 = 0.0625 + 0.04 = 0.1025$; çarpan $1 - 0.2 \cdot 0.1025 = 0.9795$; $(0.2449,\ 0.1959)$. Kayma $500 \cdot \sqrt{0.0051^2 + 0.0041^2} \approx 3.3$ piksel.

P2

P2.1 8 serbestlik, 4 nokta. Saf dönmede $\mathbf{x}_2 \sim \mathbf{K}\mathbf{R}\mathbf{K}^{-1}\mathbf{x}_1$: derinlikten bağımsız, yani bir homografi.

P2.2 $k = \log 0.01/\log(1 - 0.5^4) = 72$; $s = 8$: $\log 0.01/\log(1 - 0.5^8) = 1177$.

P2.3 Noktaları merkezi sıfır, ortalama uzaklığı $\sqrt{2}$ olacak şekilde öteler ve ölçekler. Ham piksellerde $\mathbf{A}$ matrisinde 10^0 ile 10^5 mertebesi elemanlar karışır, SVD kötü koşullanır; gürültü sonucu büyütülerek bozar (Bölüm 3: 84.7° yerine 6.6°).

P3

P3.1 $\mathbf{E}$: 5, $\mathbf{F}$: 7, $\mathbf{H}$: 8. $\mathbf{E}$'nin tekil değerleri $(\sigma, \sigma, 0)$.

P3.2 $\mathbf{t}$ ve sahne birlikte ölçeklenince görüntüler değişmez. Ölçek: bacak odometrisi (adım uzunluğu kinematikten bilinir), IMU (ivme metre cinsinden), bilinen boyutlu bir işaret (AprilTag kenar uzunluğu), stereo.

P3.3 $z = fb/d = 500 \cdot 0.1/10 = 5$ m; $\sigma_z = z^2\sigma_d/(fb) = 25 \cdot 0.5/50 = 0.25$ m. 2 m'de: $4 \cdot 0.5/50 = 0.04$ m.

P4

P4.1 $\bar{\mathbf{p}} = (1, 0)$, $\bar{\mathbf{q}} = (1, 2)$; merkezlenmiş: $\mathbf{p}' = (\mp 1, 0)$, $\mathbf{q}' = (0, \mp 1)$. x ekseni y eksenine gidiyor: $\mathbf{R} = \mathrm{Rot}(90^\circ) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$. $\mathbf{t} = \bar{\mathbf{q}} - \mathbf{R}\bar{\mathbf{p}} = (1, 2) - (0, 1) = (1, 1)$. Sağlama: $\mathbf{R}(2, 0) + \mathbf{t} = (0, 2) + (1, 1) = (1, 3)$.

P4.2 En yakın komşu eşleşmeleri ancak tahmin gerçeğe yakınsa doğrudur; ICP yerel bir iniş

yöntemi. Dikdörtgen oda 90° döndürülünce kendine çok benzer (kolon dışında); 45°'nin ötesinde duvarlar yanlış duvarlara daha yakın kalır ve ICP o yerel minimuma iner.

P4.3 Haritada karşılığı olmayan (yeni görülen, örtülü) noktaların yanlış eşleşmelerini atar. Çok küçük: başta doğru eşleşmeler bile atılır, yakınsama başlamaz. Çok büyük: aykırılar Kabsch'ı çeker, sonuç kayar.

P5

P5.1 $3 + 2 \cdot 50 = 103$; $103^2 = 10\,609$ eleman.

P5.2 $K = P/(P+R) = 0.8$; $\hat{x}^+ = 10 + 0.8 \cdot 2 = 11.6$; $P^+ = (1-0.8) \cdot 4 = 0.8$.

P5.3 Robotun pozu ile gördüğü her işaret arasında kovaryans terimleri birikir. Eski işareti görmek robotu düzeltir; Kalman kazancı bu korelasyonlar üzerinden düzeltmeyi bağlı tüm işaretlere yayar. (EKF geçmiş pozları tutmaz ama onlarla ilişkili işaretleri düzeltir; poz grafiği geçmiş pozları da açıkça düzeltir.)

P6

P6.1 $3 \cdot 147 - 3 \cdot 120 = 81$. 2000: ya gürültü modelleri çok iyimser ($\mathbf{\Omega}$ fazla büyük) ya da yanlış bir döngü kapatma var; tek tek kenar hatalarına bak, robust kayıp kullan.

P6.2 Tüm pozları birlikte aynı miktarda kaydırmak/döndürmek göreli ölçümlerin hiçbirini değiştirmez; maliyet o yönde sabit $\Rightarrow$ $\mathbf{H}$'nin 3 boyutlu sıfır uzayı var.

P7

P7.1 $\frac{1}{2} \cdot 0.15 \cdot 10^2 = 7.5$ m; 20 s: 30 m. Simülasyonda filtre hızı 0'dan başlattı ama gerçek hız 0.3 m/s idi: bu ters yönde $0.3 \cdot 20 = 6$ m hata ekler; $30 - 6 \approx 24$ m.

P7.2 Ayak sabit, bacak tepesi (kalça) ayağın etrafında döner: $v = \ell\dot{q} = 0.3 \cdot 1 = 0.3$ m/s (yön dönme yönüne göre).

P7.3 $S = 0.04^2 + 10^{-4} = 0.0017$; $d^2 = 0.35^2/0.0017 = 72$. $72 > 9$: reddedilir.